

An aerial photograph of the University of Colorado Boulder campus. The foreground is filled with trees showing vibrant autumn foliage in shades of yellow, orange, and red. In the middle ground, several university buildings are visible, including a prominent red brick building with a tall, ornate clock tower. An American flag flies from a tall pole on the roof of the clock tower. The background features a large, rugged mountain with rocky peaks under a clear blue sky.

Training Systems

CSCI 5942

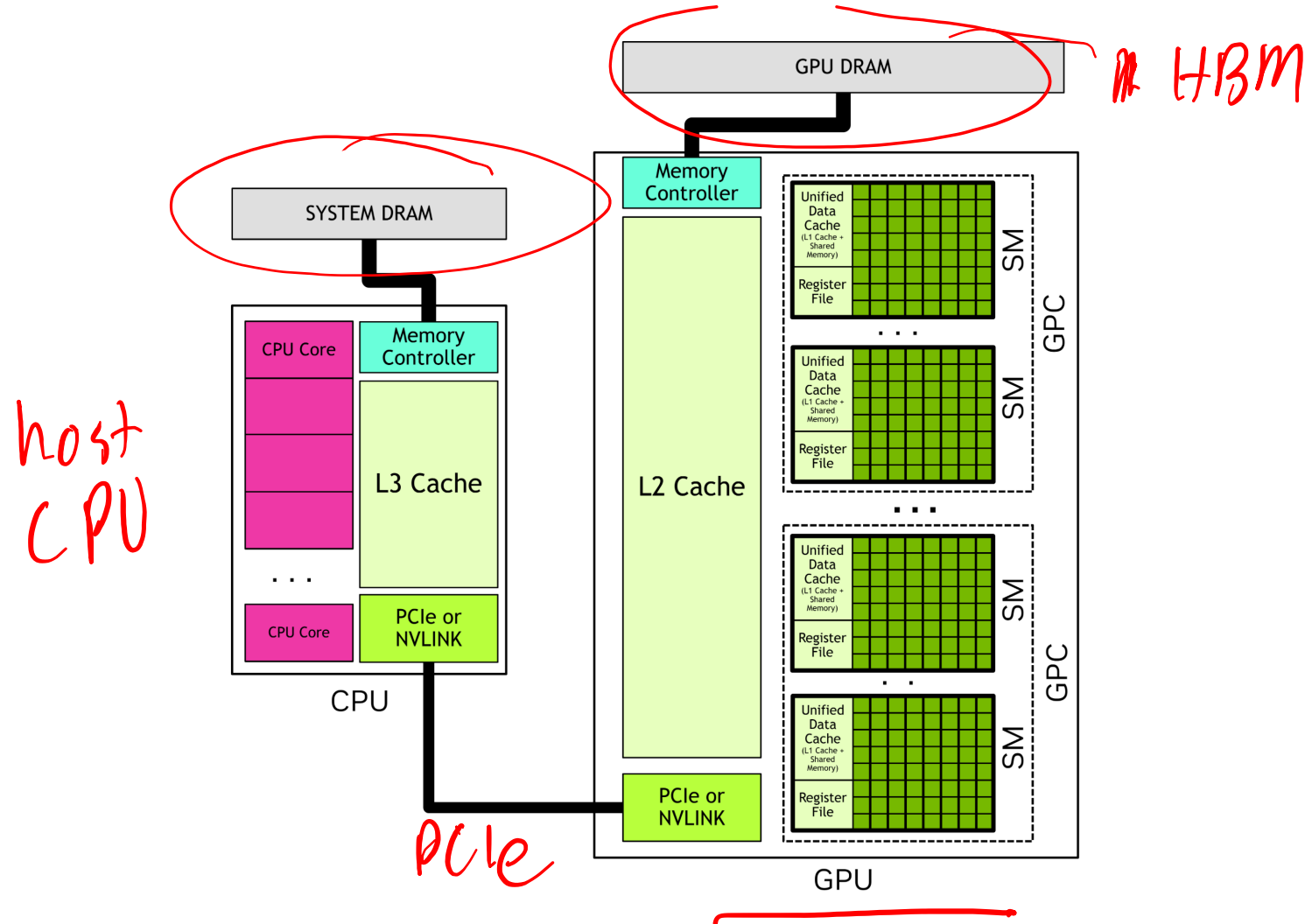
Lecture 7



College of Engineering & Applied Science

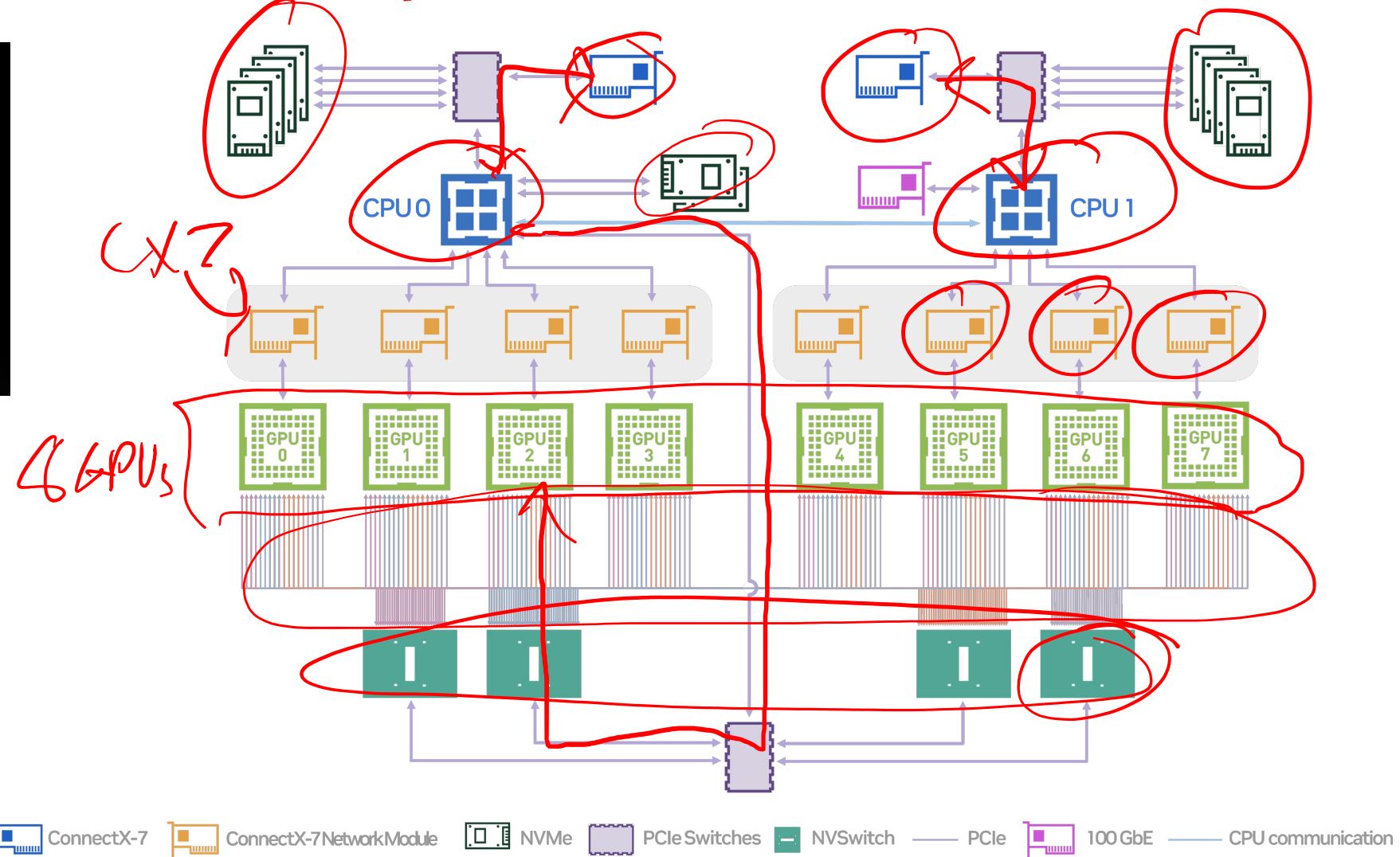
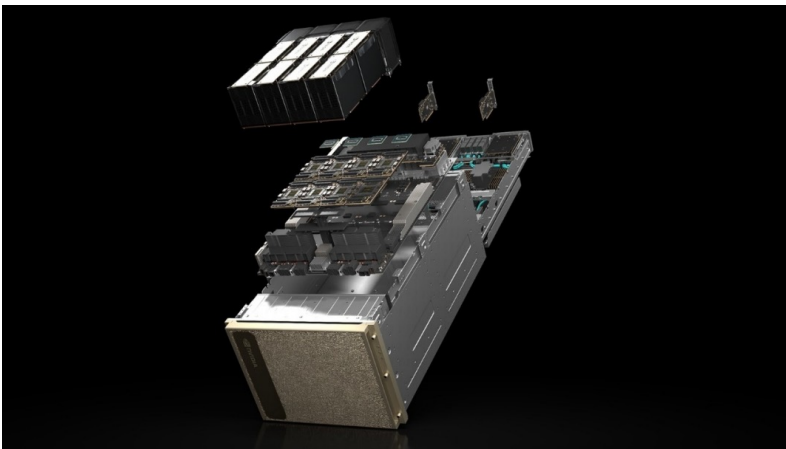
UNIVERSITY OF COLORADO **BOULDER**

Recap: The GPU



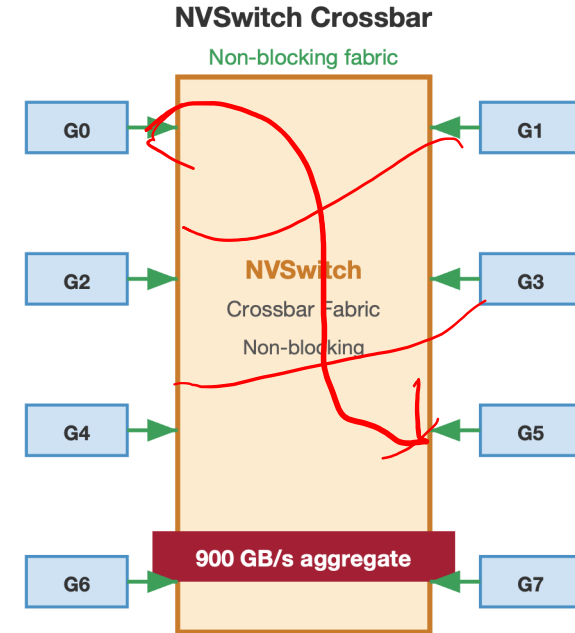
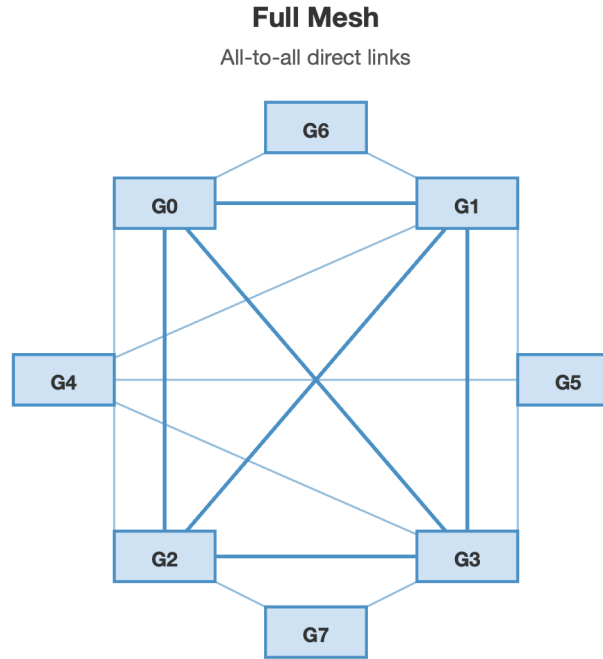
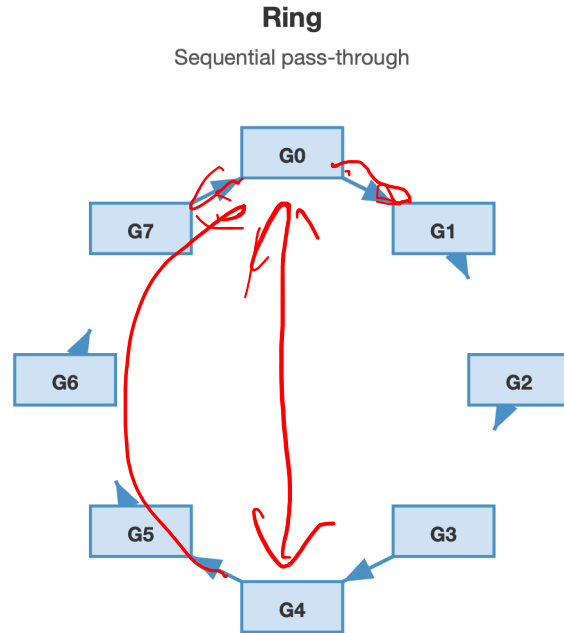
The GPU Node

Storage SSDs DGX H100



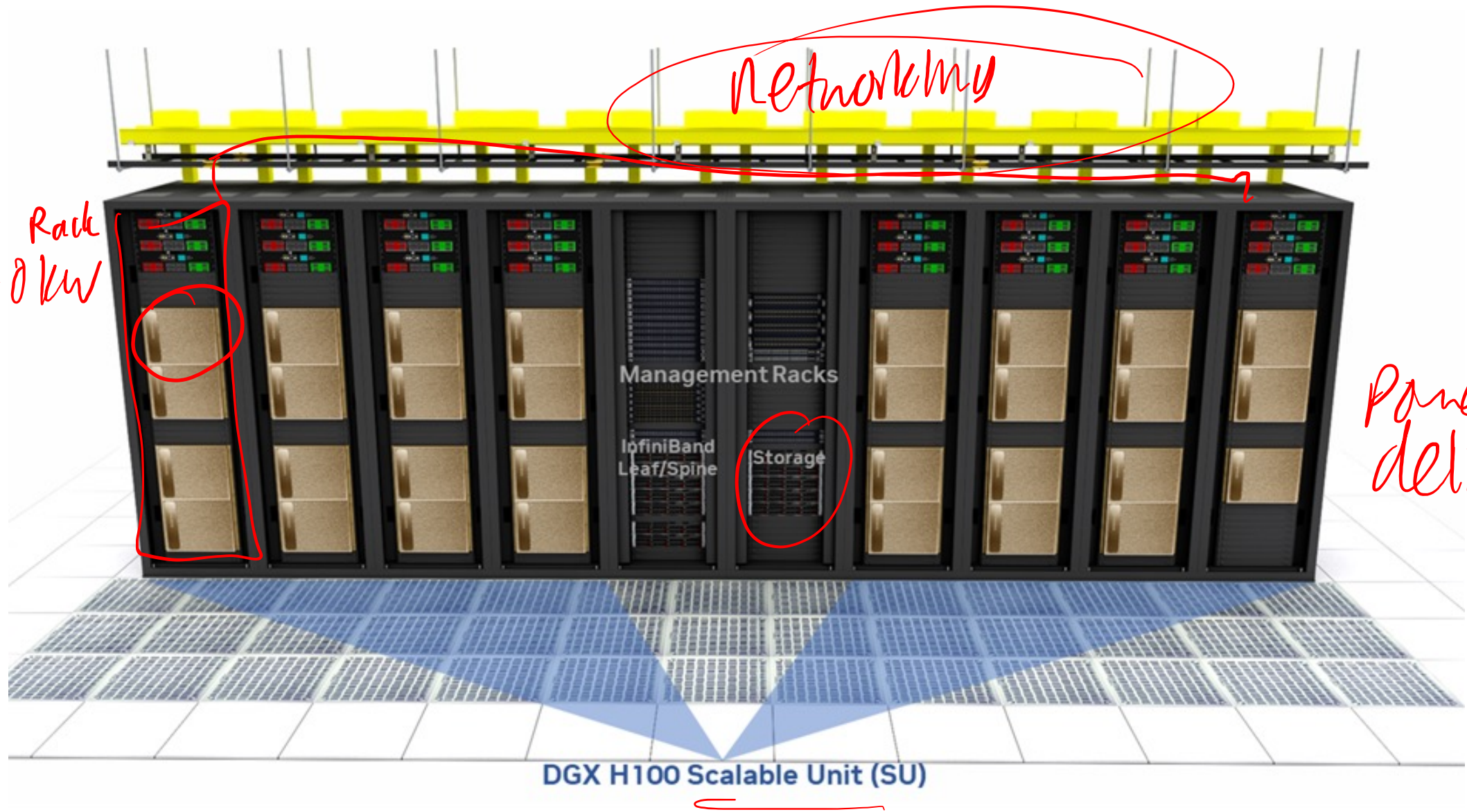
Intra-node / “Scale-Up” Interconnect

Intra-Node GPU Topology Comparison

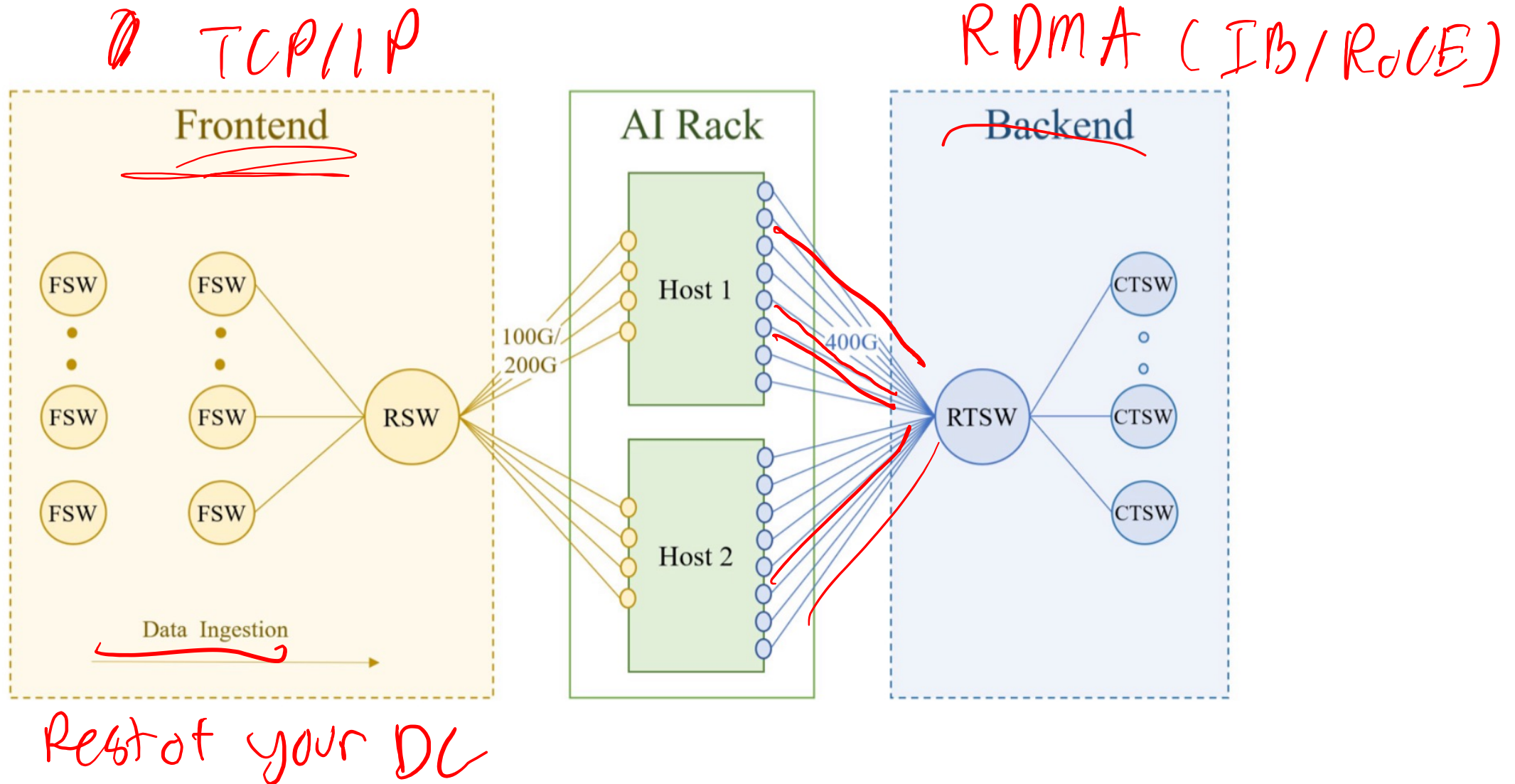


100 GB/s
bidirectional GPU-GPU
bandwidth

The Rack / “Pod”



Inter-node / “Scale-out” Network

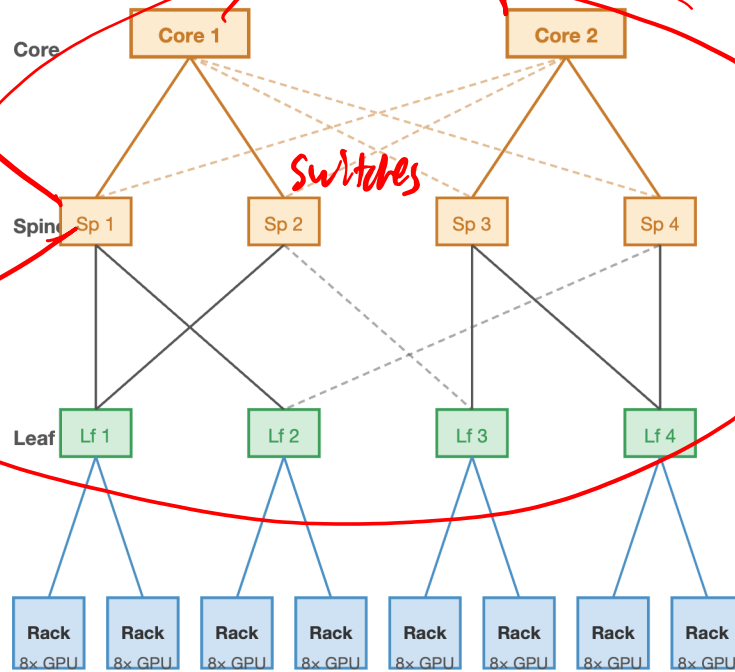


Inter-node / “Scale-out” Network

Pod-Scale Network Topologies

Fat-Tree (3-Tier)

Used by InfiniBand clusters (DGX SuperPOD)

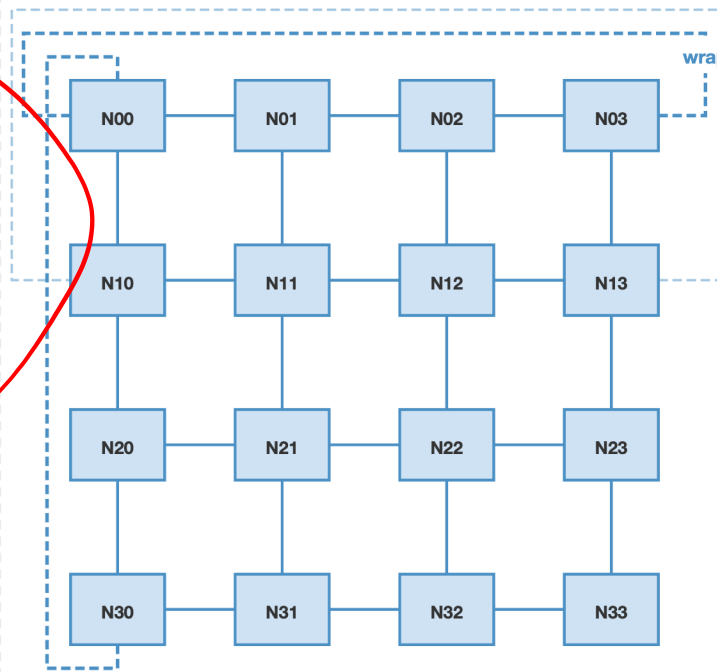


Bisection bandwidth: full (non-blocking)

Each path has dedicated capacity — no contention

2D Torus

Used by Google TPU pods (mesh wrap-around)

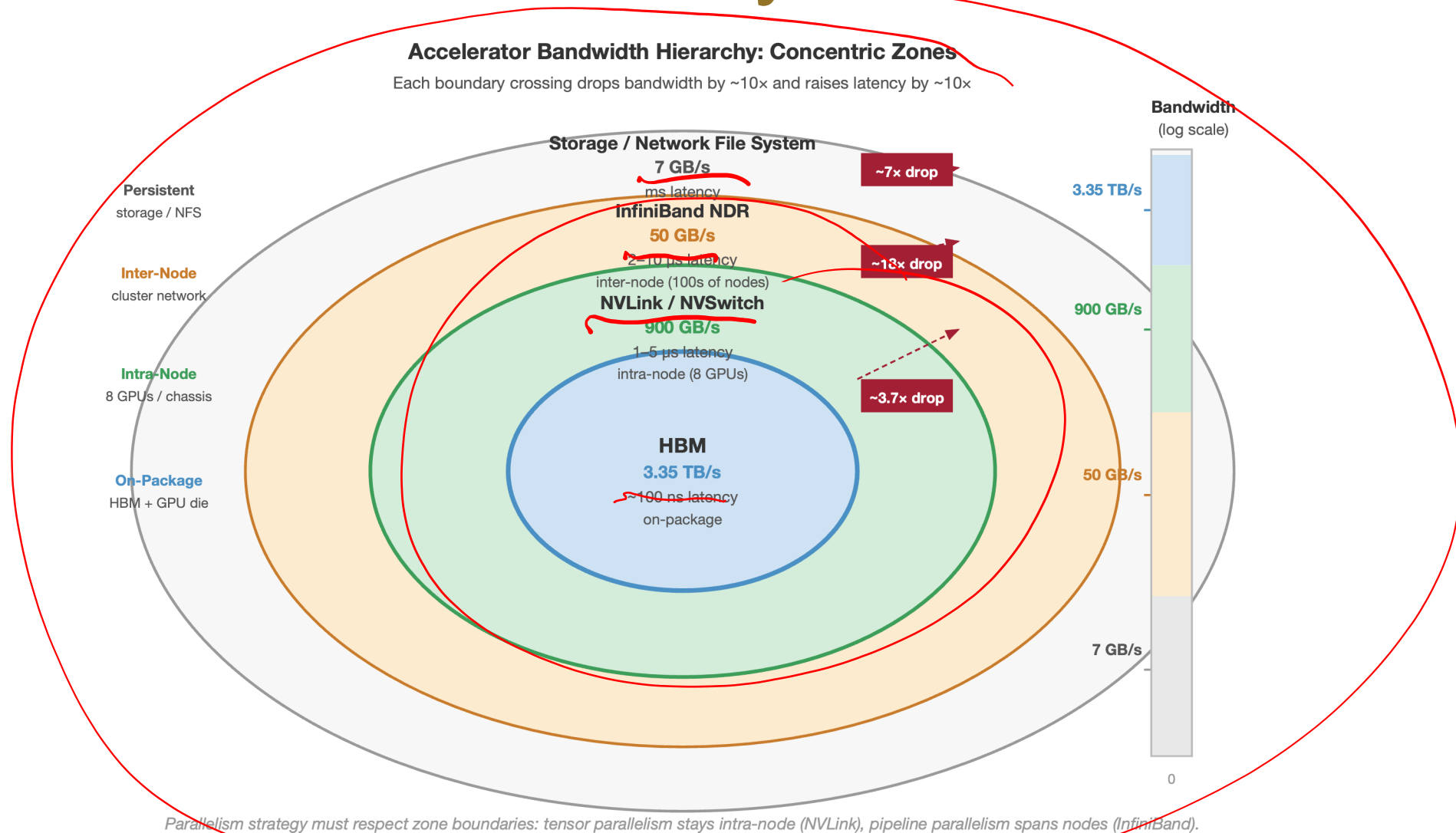


Bisection bandwidth scales with $\sqrt{N}$

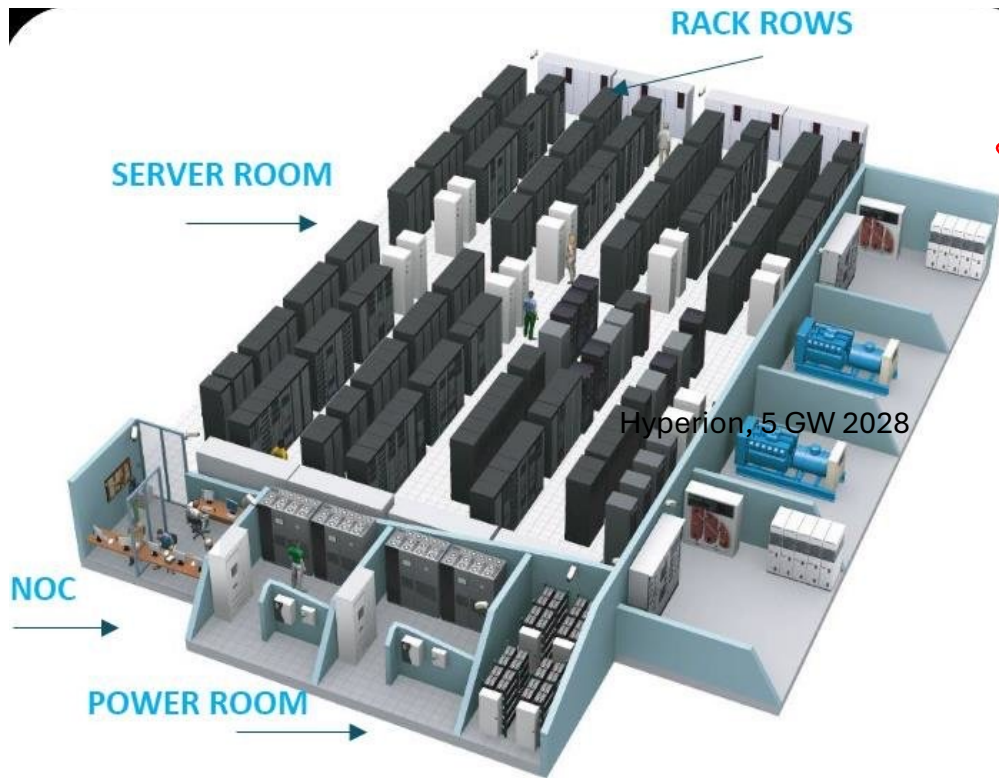
All N→N traffic must cross the bisection cut
Locality of workload placement is critical

Fat-tree provides full bisection bandwidth for any traffic pattern; torus is cheaper but rewards workloads with strong locality.

The Bandwidth Hierarchy



Datacenters and Regions



BUILDING 1

BUILDING 2

CENTRAL UTILITY
BUILDING &
WATER TANK

BUILDING 3

SUBSTATION 1

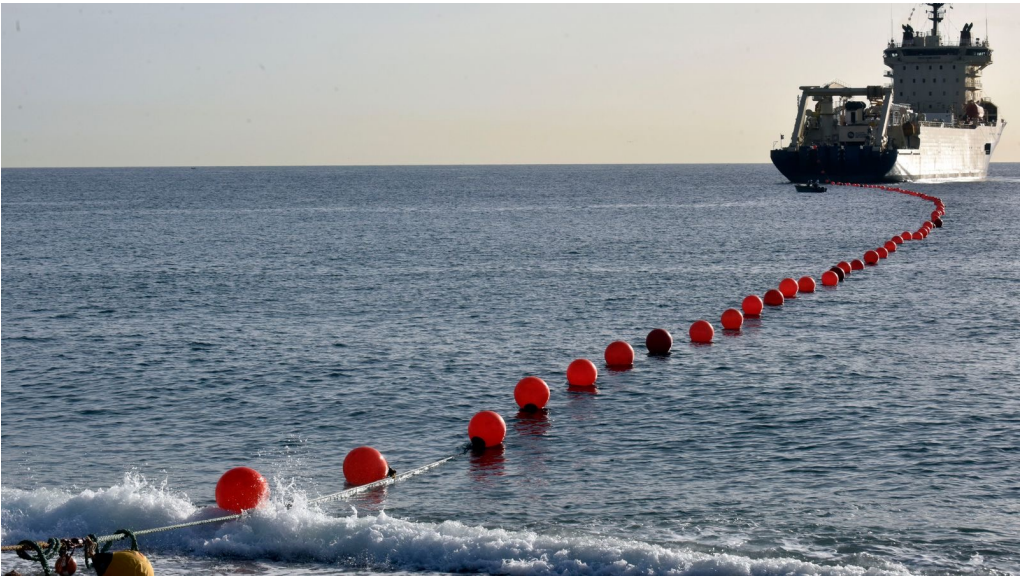
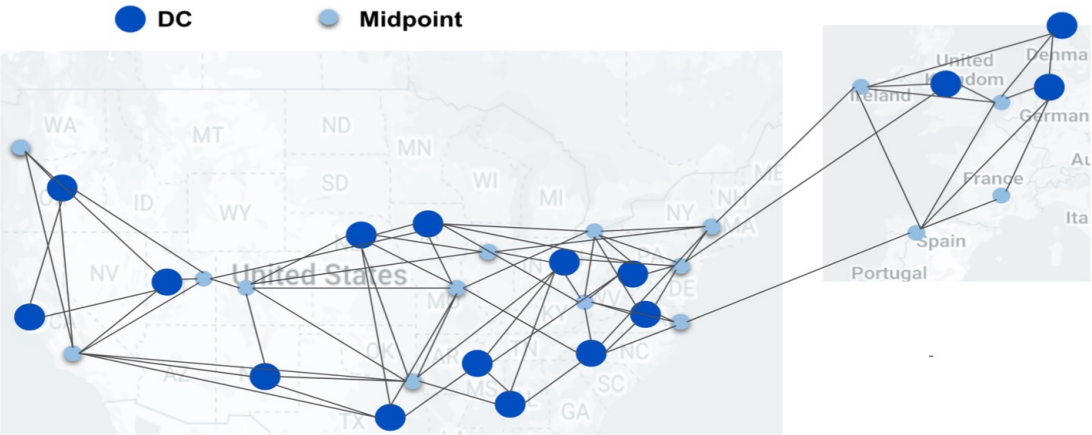
SUBSTATION 2



A region-wide training cluster

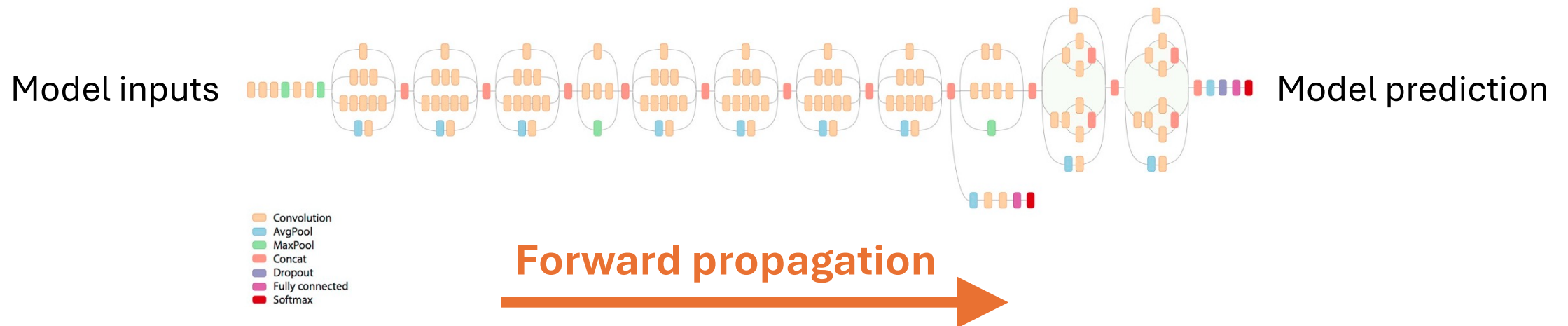


Global Regions



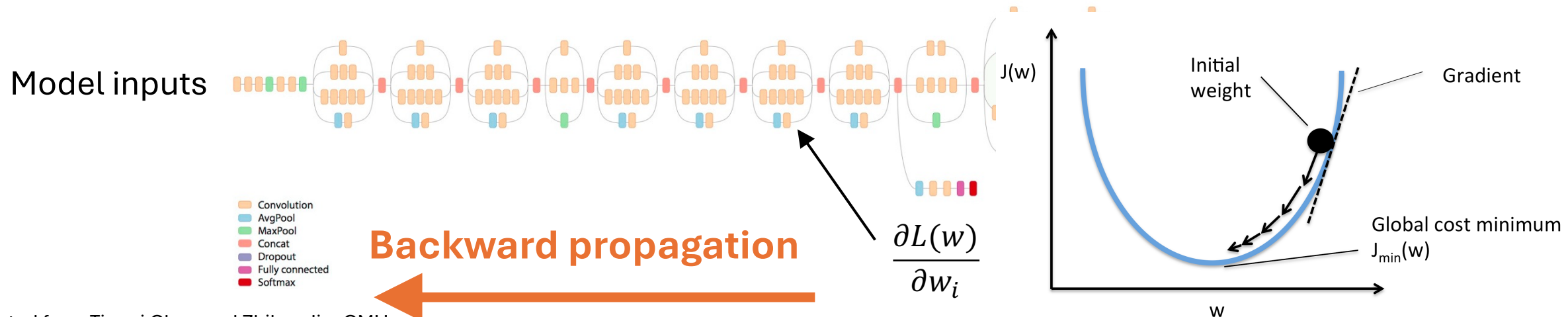
Recap: DNN Training

- Forward propagation: Apply model to a batch of input samples and run calculation through operators to produce a prediction



Recap: DNN Training

- Forward propagation: Apply model to a batch of input samples and run calculation through operators to produce a prediction
- Backwards propagation: Run the model in reverse to produce a gradient for each trainable weight



Recap: DNN Training

- Forward propagation: Apply model to a batch of input samples and run calculation through operators to produce a prediction
- Backwards propagation: Run the model in reverse to produce a gradient for each trainable weight
- Weight update: Use the gradients to update model weights

$$w_i := w_i - \gamma \frac{\partial L(w)}{\partial w_i} = w_i - \frac{\gamma}{n} \sum_{j=1}^n \boxed{\frac{\partial l_i(w)}{\partial w_i}}$$

Gradients of individual samples

Why do we want more GPUs?

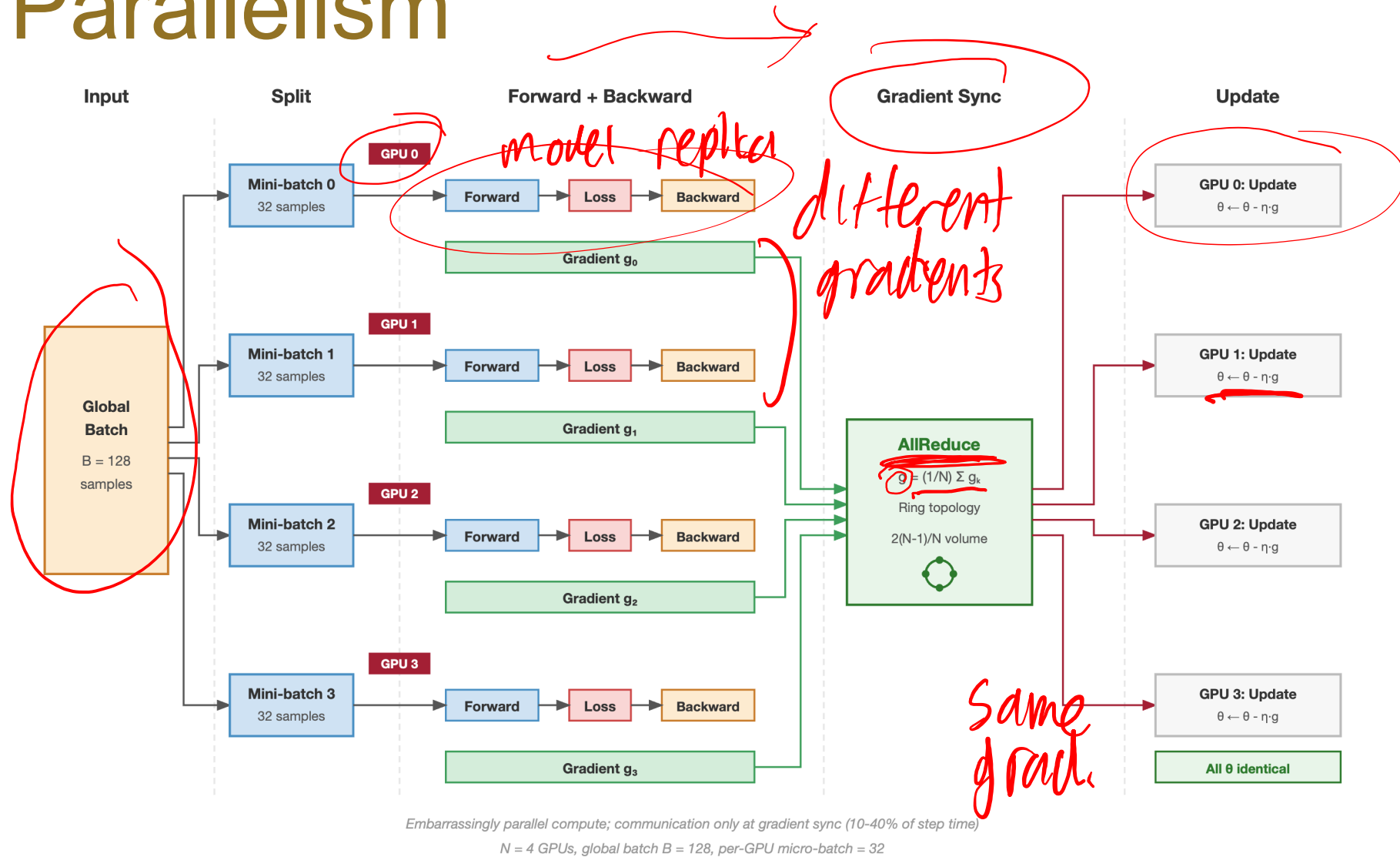
Faster: More samples/second

Larger: - Batch size
- model size

Data Parallelism

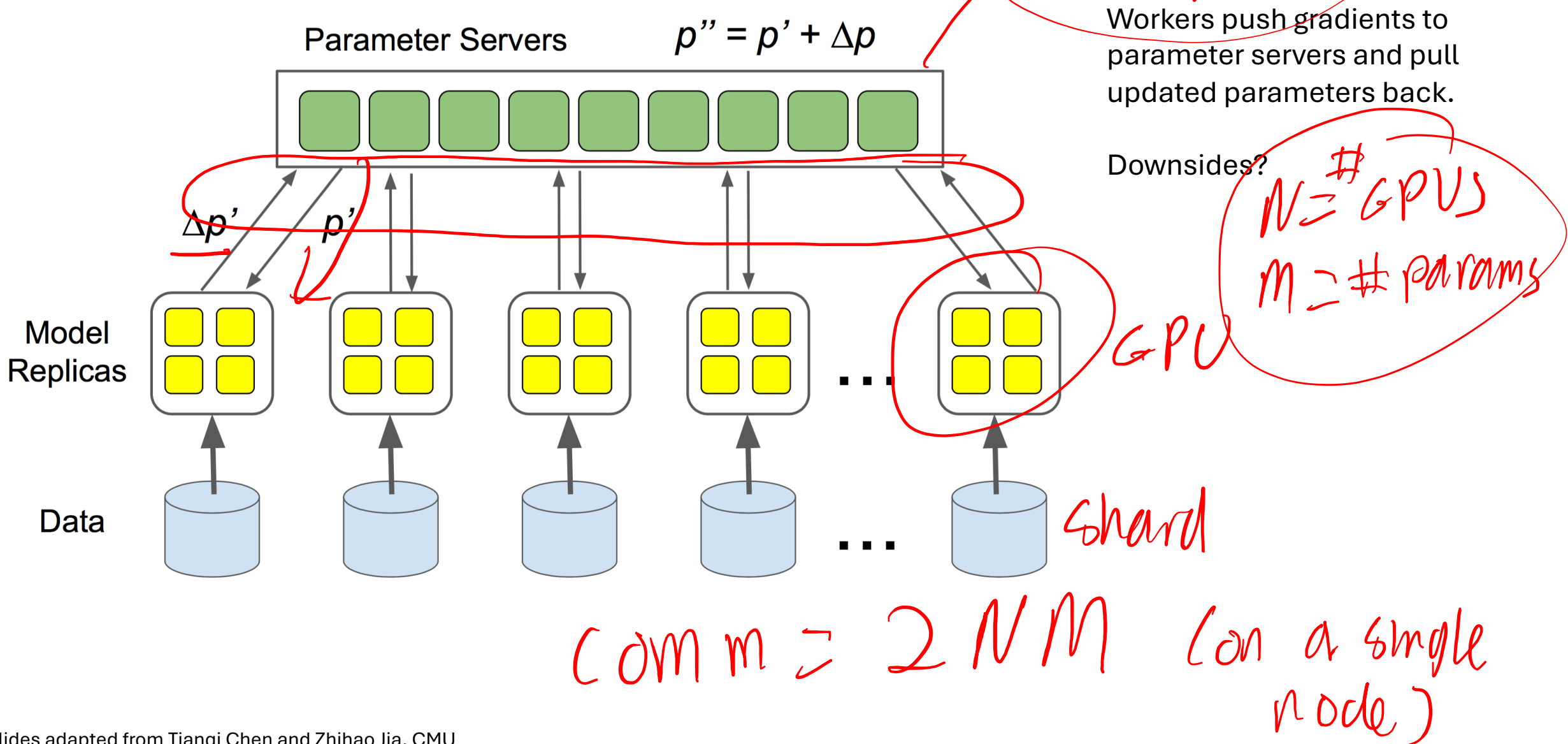
- Goal: Scale compute (i.e., process more data faster)
- Key idea: Partition training *data* across many GPUs

Data Parallelism



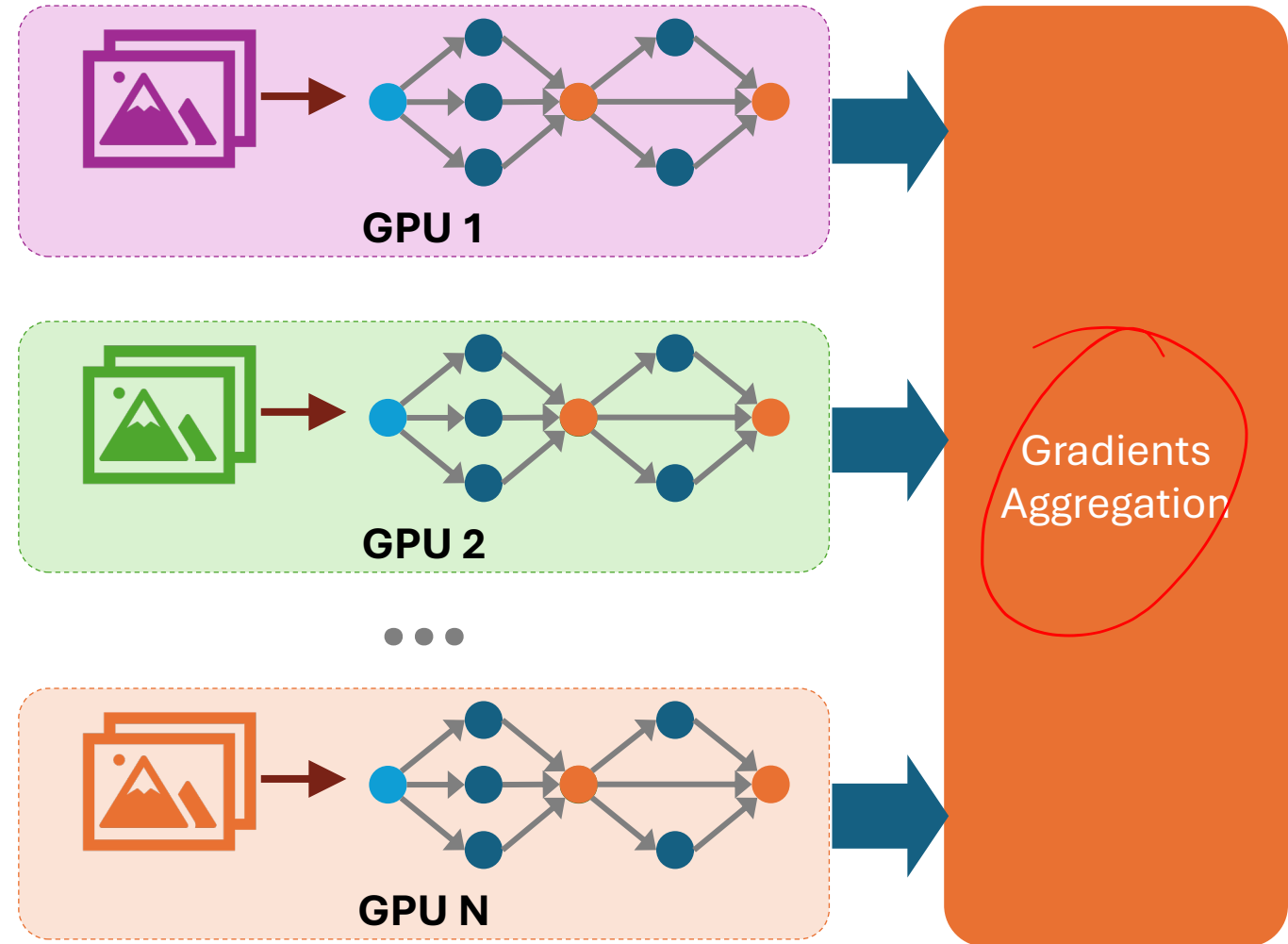
Key question: How do we synchronize gradients across all parallel workers?

Option 1: Parameter Servers



Synchronous Data Parallelism

- *Replicate* model on each GPU
- For each iteration:
 - *Partition* data across GPUs
 - Each GPU: Fwd/Bwd pass
 - Generate gradients unique to each GPU
 - *Sync gradients (e.g., avg) across all GPUs*
 - Update weights (synced) on all GPUs



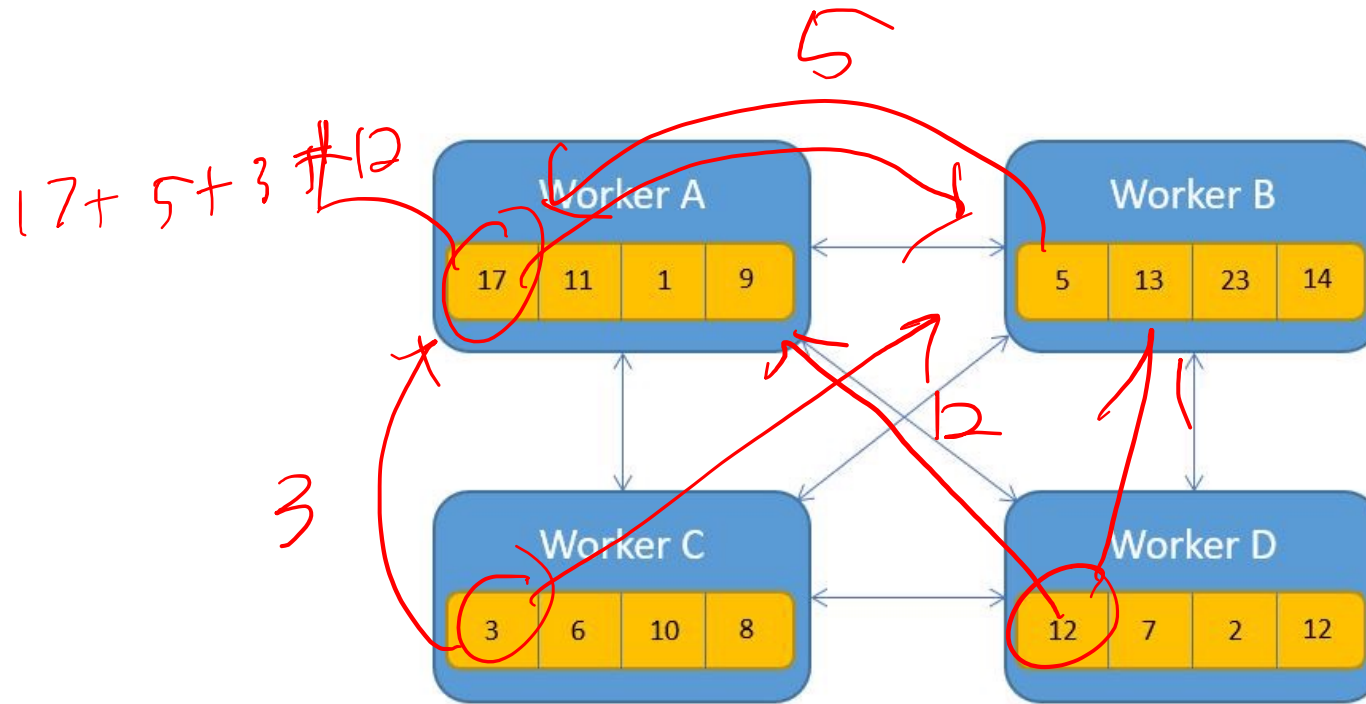
All Reduce

- Goal: Perform element-wise reduction across multiple devices



Naïve AllReduce

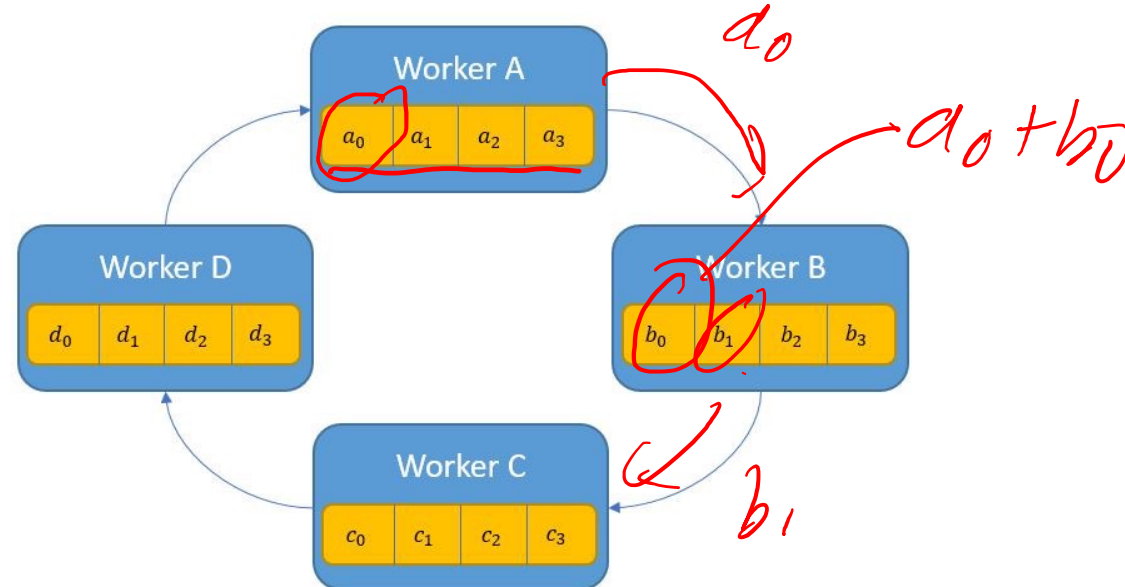
N workers
 M params



$$\underbrace{O((N-1)M)}_{O(N^2M)}$$

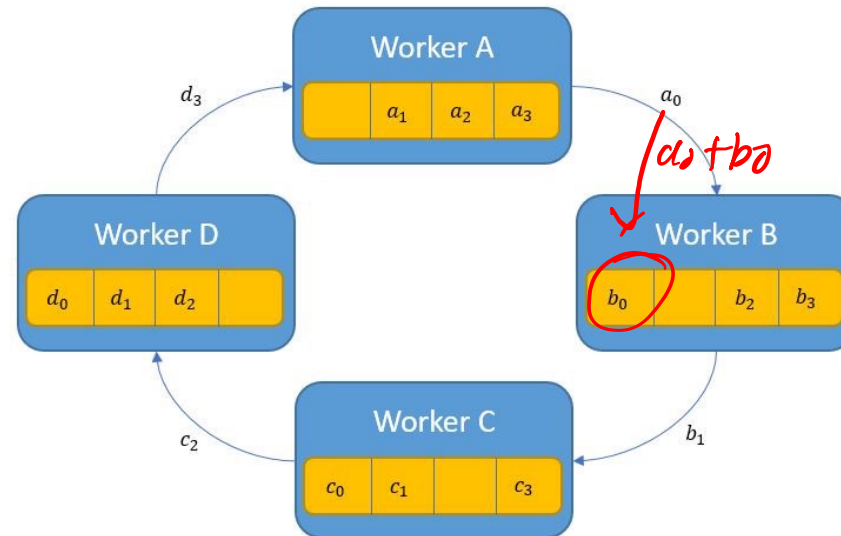
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
 - Each worker sends one slice (M/N params) to the next worker on the ring
 - Worker reduces its received slice with current slice
 - Repeat N times



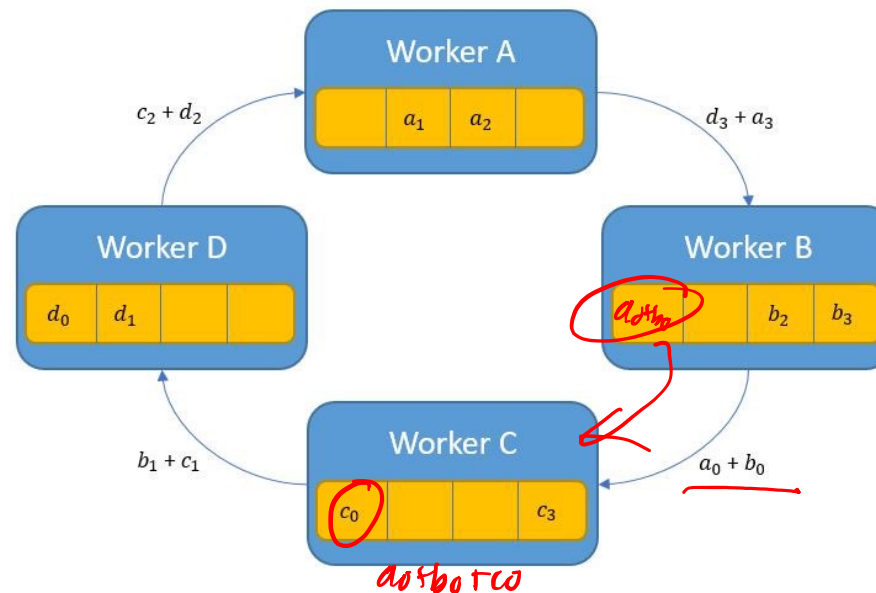
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
 - Each worker sends one slice (M/N params) to the next worker on the ring
 - Worker reduces its received slice with current slice
 - Repeat N times



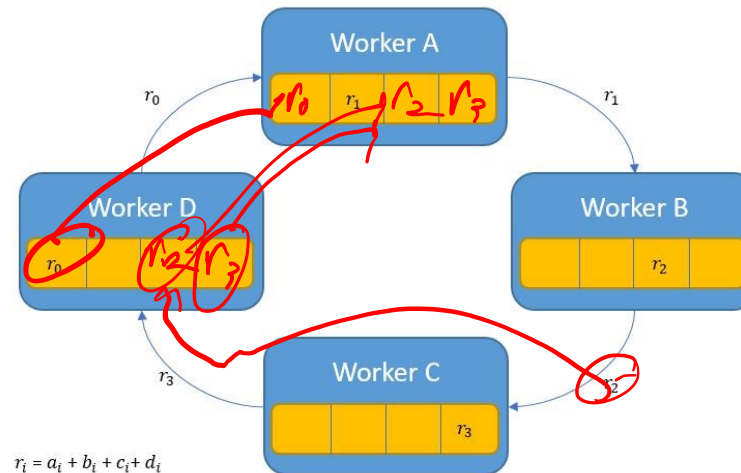
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
 - Each worker sends one slice (M/N params) to the next worker on the ring
 - Worker reduces its received slice with current slice
 - Repeat N times



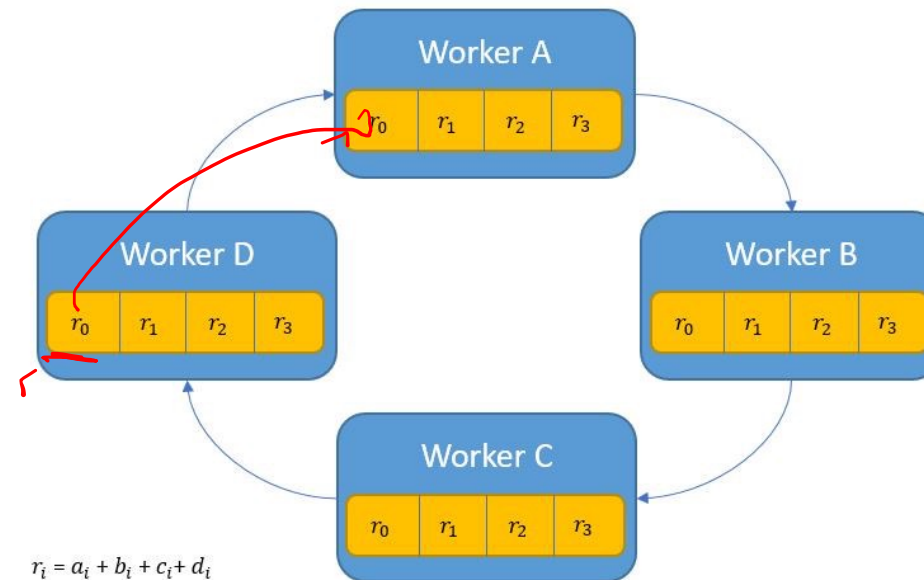
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
 - Each worker sends one slice (M/N params) to the next worker on the ring
 - Worker reduces its received slice with current slice
 - Repeat N times
 - End up with reduced version of one slice



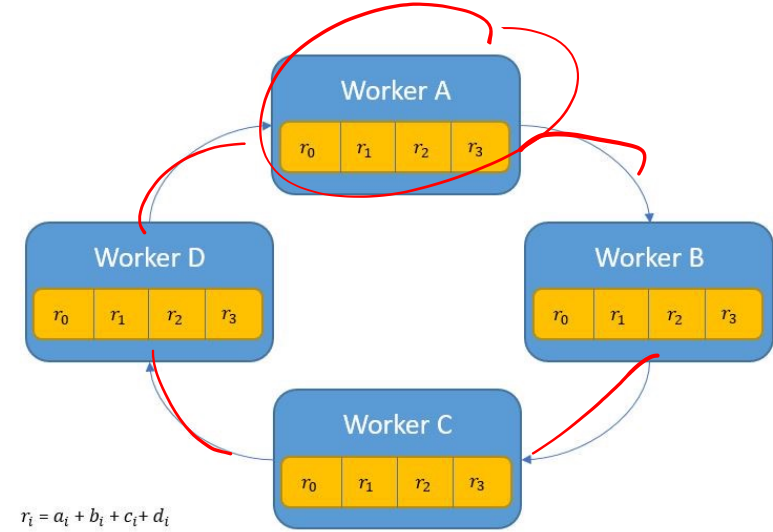
Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
- Step 2 (Broadcast):
 - Each worker sends one slice of reduced parameters to the next worker
 - Repeat N times



Ring AllReduce

- Construct a ring of N workers, divide M parameters into N slices
- Step 1 (Aggregation):
 - Each worker sends one slice (M/N params) to the next worker on the ring
 - Worker reduces its received slice with current slice
 - Repeat N times
- Step 2 (Broadcast):
 - Each worker sends one slice of reduced parameters to the next worker
 - Repeat N times



Agg! $\frac{M}{N} \cdot N \cdot N = MN$

Broadcast $= MN$

Total $= O(MN)$
parameters

Param Server $= M \cdot N \cdot 2$

Issues with Data Parallelism

- Each GPU needs to store entire model!
- Ex: Training a 7B parameter Llama model in mixed precision. Does the full training state fit on an H100-80GB GPU?

Weights FP16 = 2 B

Grads FP16 = 2 B

Adam optimizer

– Master weights FP32 = 4 B

– moment = 4 B

variance = 4 B

16 Bytes /

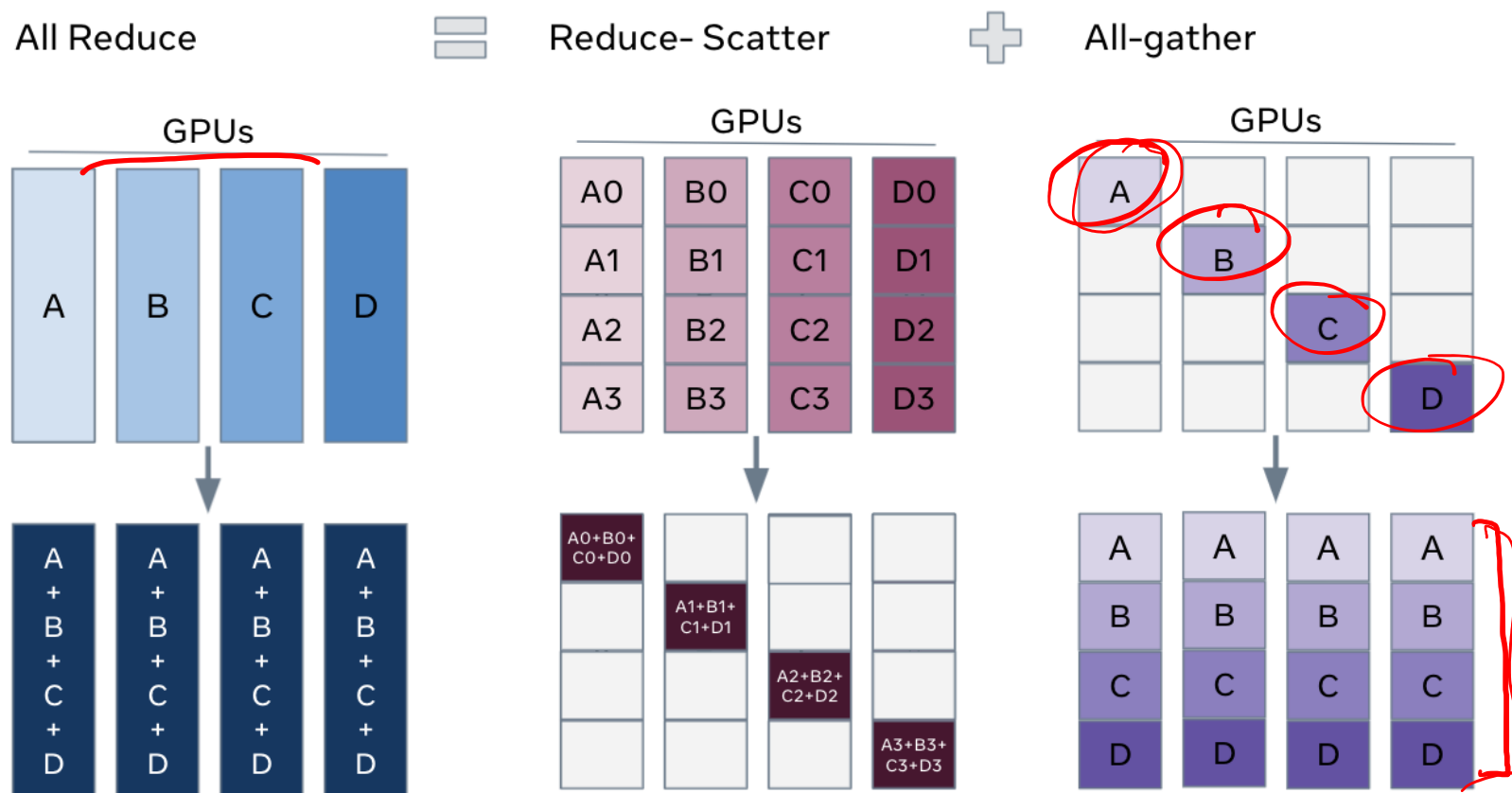
Param.

= 112 GB

Model Parallelism

- Goal: Scale **parameters (memory)**
- Key idea: Shard **model** across GPUs
- Two main flavors:
 - Tensor Parallelism
 - Pipeline Parallelism

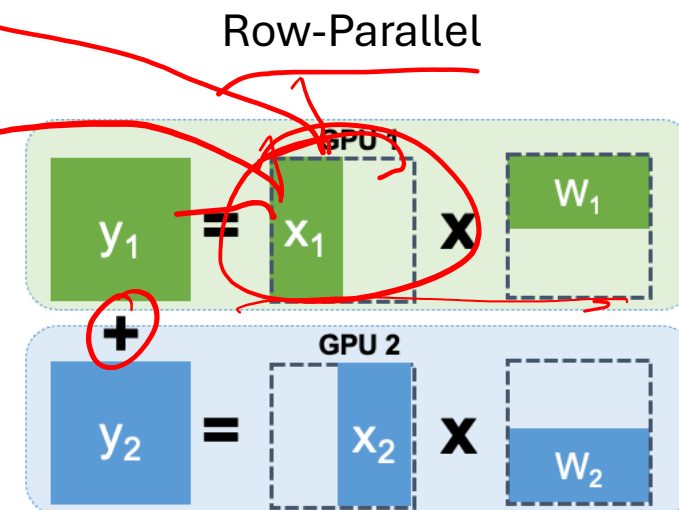
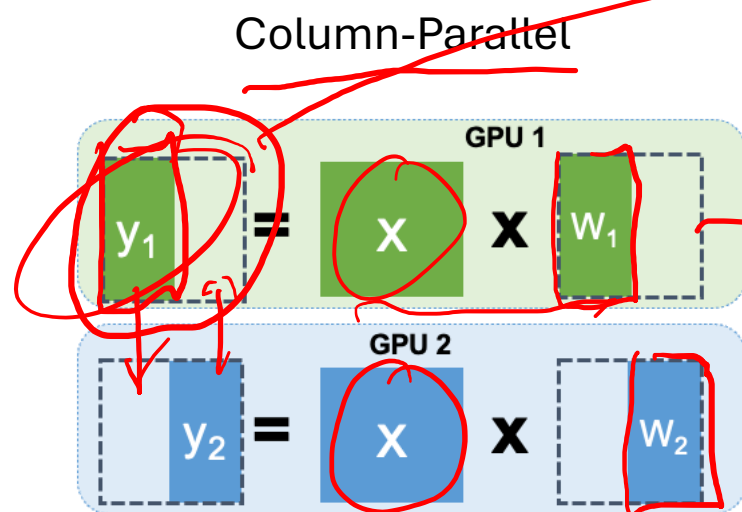
ReduceScatter, AllGather



Tensor Parallelism

$$Y = XW$$

- Shard *model weights* across GPUs



$$Y = XW = X[W_1 | W_2] = [XW_1 | XW_2]$$

Fwd! All-gather

Bwd! all-reduce

$$Y = XW = X[W_1 + X_2 W_2]$$

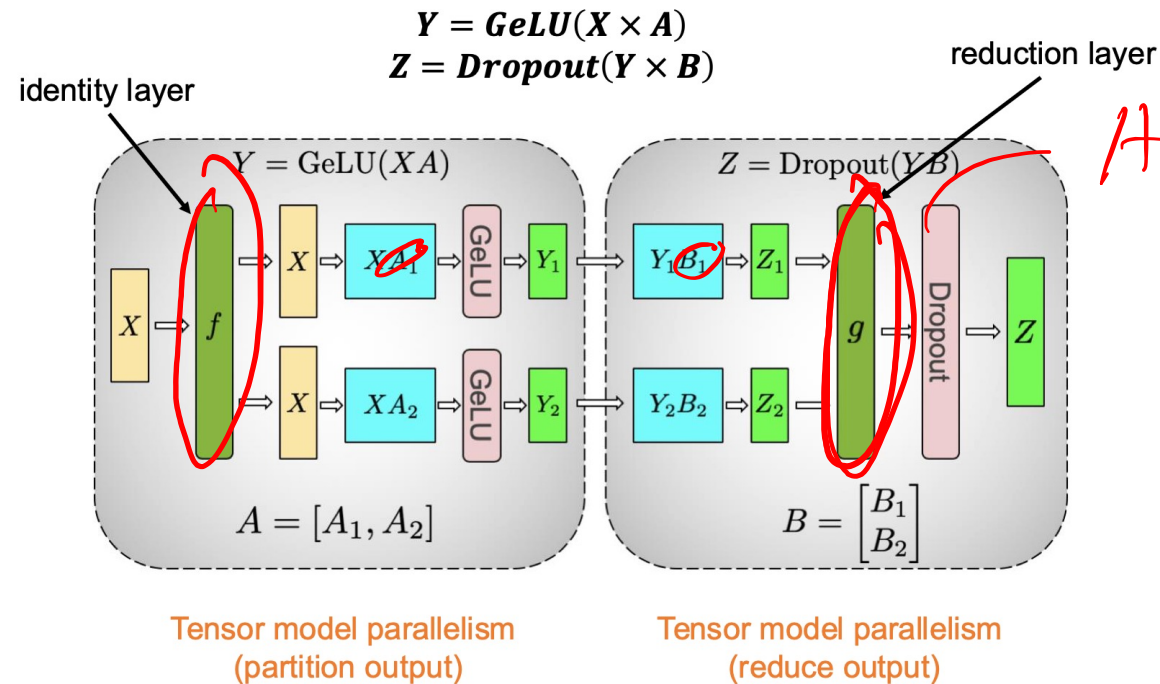
Fwd! all-reduce

Bwd! all-gather

Megatron-LM Tensor Parallelism

Idea: Column-wise partition followed by row-wise to avoid an collective between them - only need two AR (fwd, bwd)

- QKV projection (column), attention output projection (row)
- (shown below) FFN 1 (column), FFN 2 (row)



Downsides of TP?

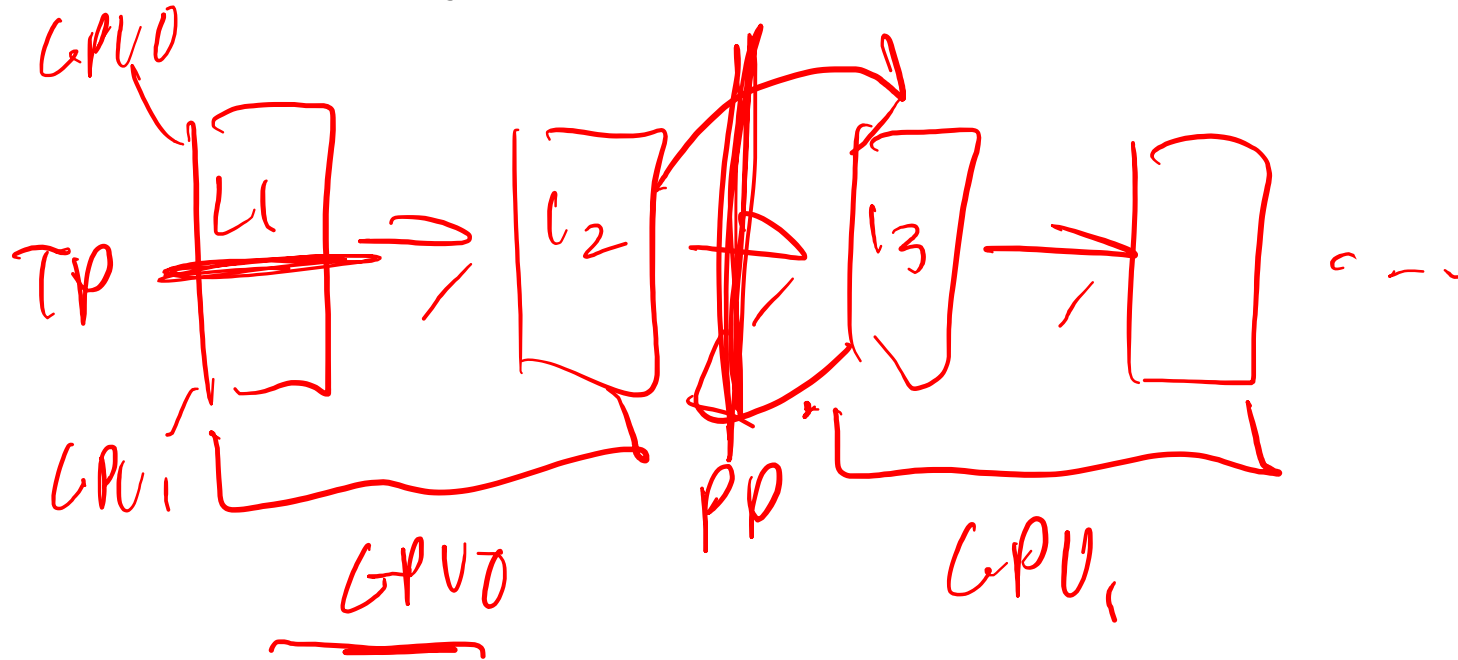
$B = 1$

- GPT-3 175B: $H = \underline{12288}$, $L = 96$ layers, $S = \underline{2048}$, microbatch = 1, bf16
- AllReduce Size = $B * S * H * 2$ bytes
 - 50 MB
- Per Layer (4 per layer)
 - 200 MB
- Per batch (96 layers)
 - **19.3 GB**

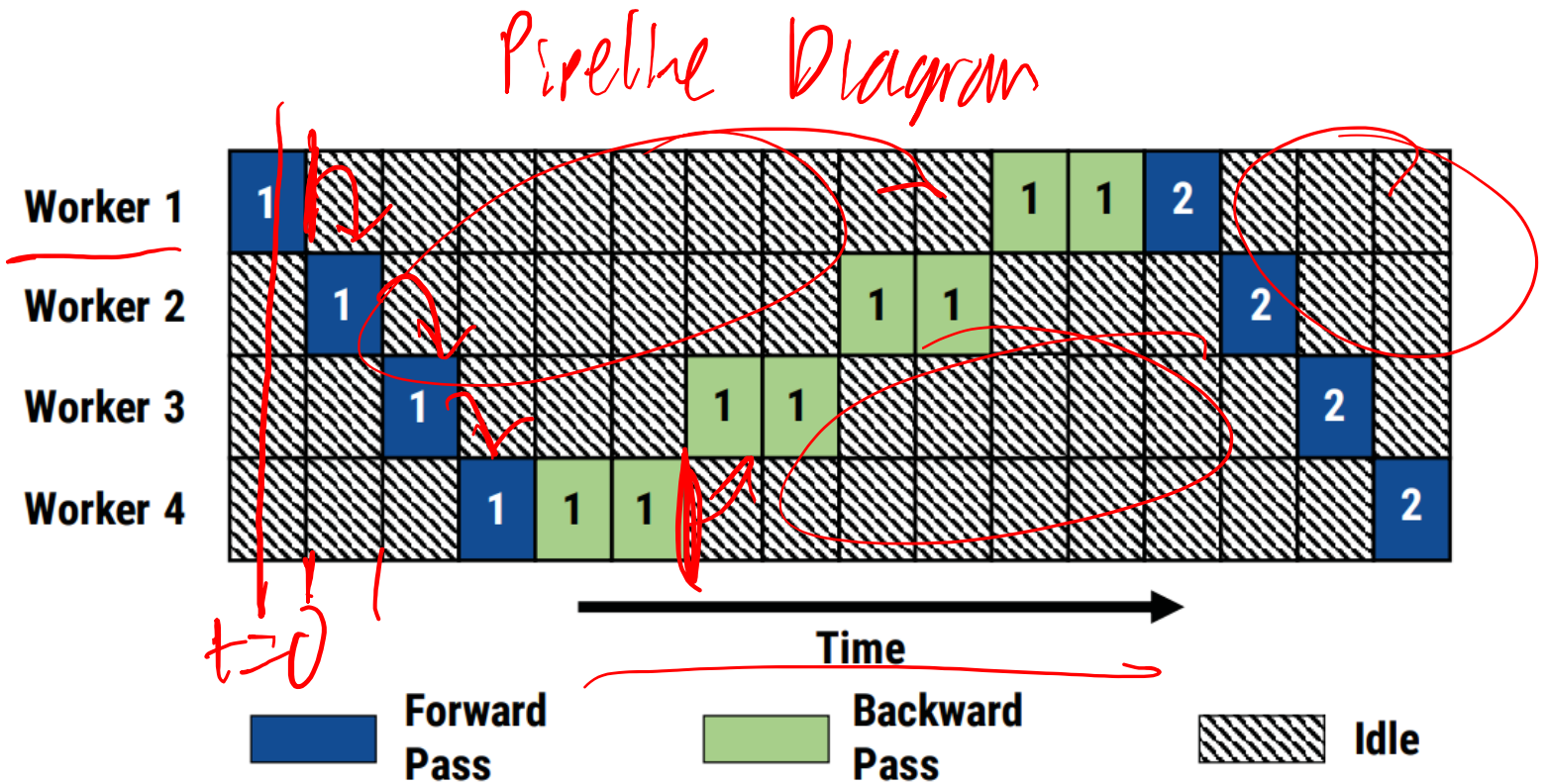
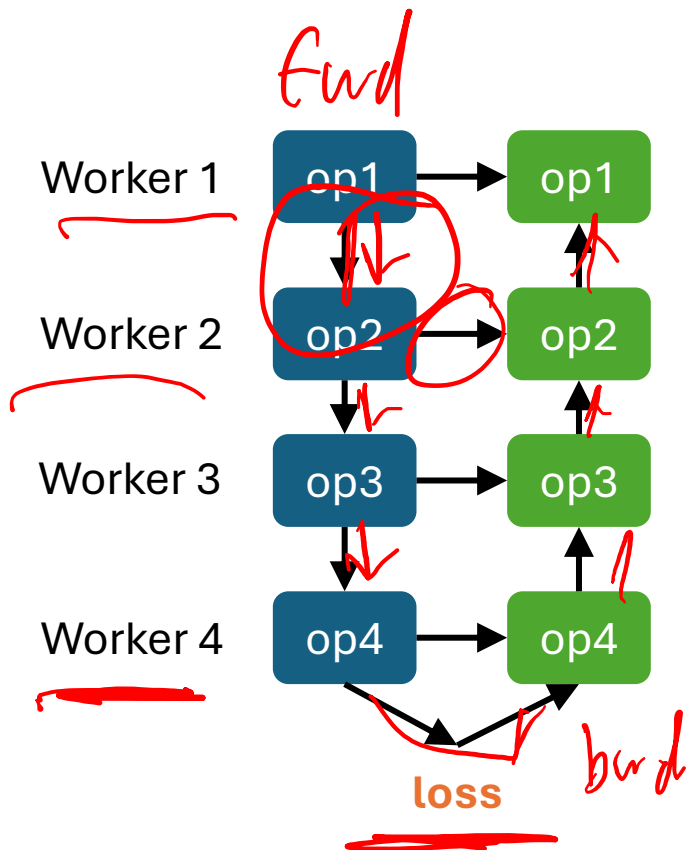
Need high network bandwidth

Pipeline Parallelism

- Goal: Reduce communication requirements between GPUs
- Idea: Shard *layers* across GPUs

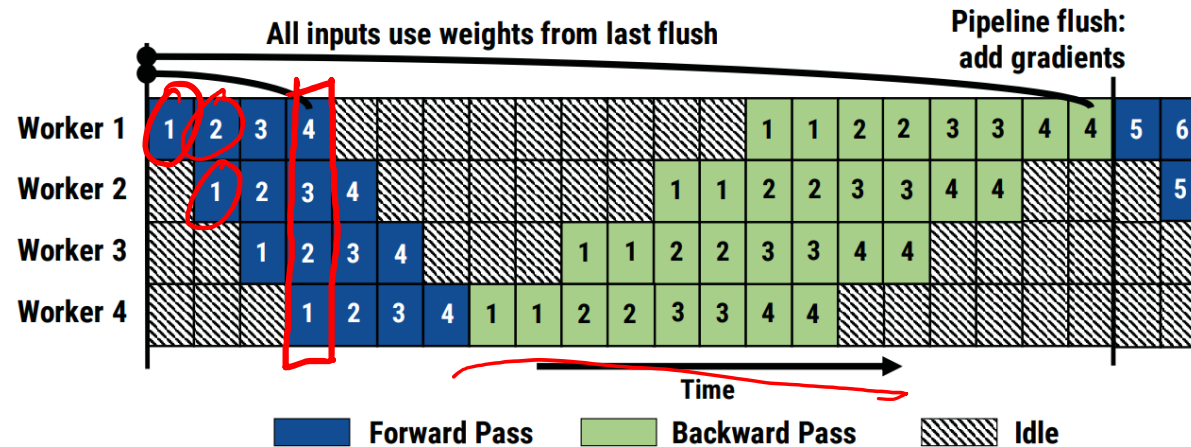


Pipeline Parallelism



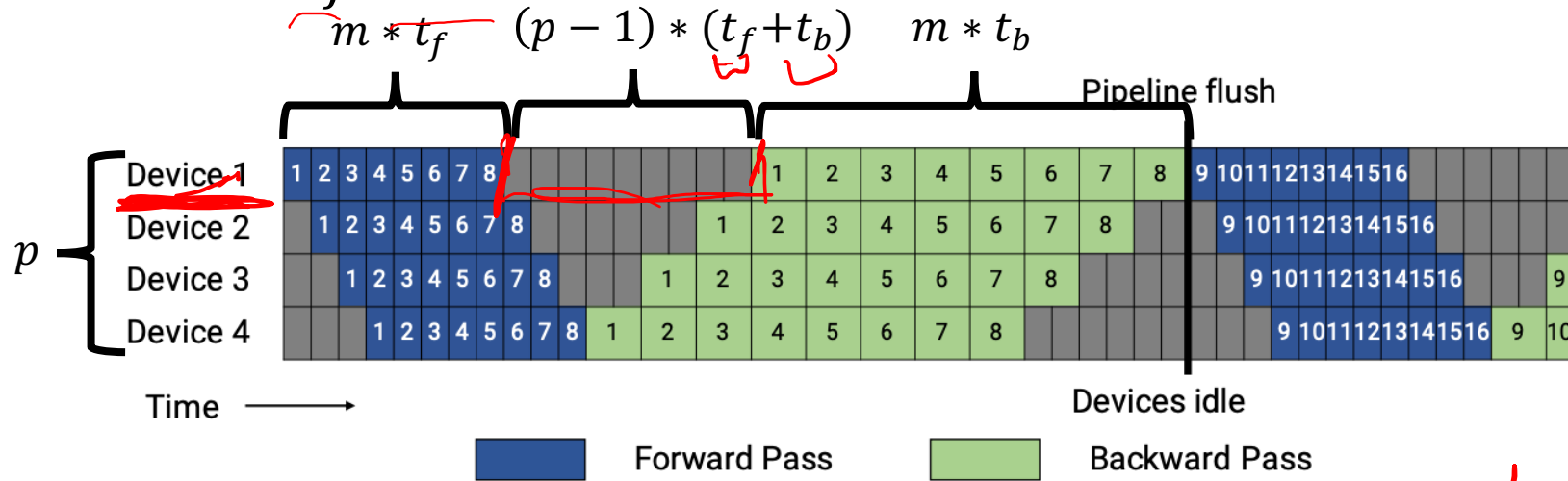
Micro-batching

- Divide mini-batch into multiple **micro-batches**
- Pipeline forward and backward computations across micro-batches



Device Utilization

- m : micro-batches in a mini-batch
- p : number of pipeline stages
- All stages take t_f / t_b to process a forward (backward) micro-batch



$$\text{Bubble fraction} = \frac{(p-1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

How to improve pipeline parallelism efficiency?

$$\text{BubbleFraction} = \frac{(p-1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p-1}{\underline{m}}$$

- 1) more micro-batches
 - more memory for activations

mini batch: 1024
micro batch = ~~10~~ 1

- 2) fewer pipeline stages
 - but requires more mem. for model state

Memory requirements

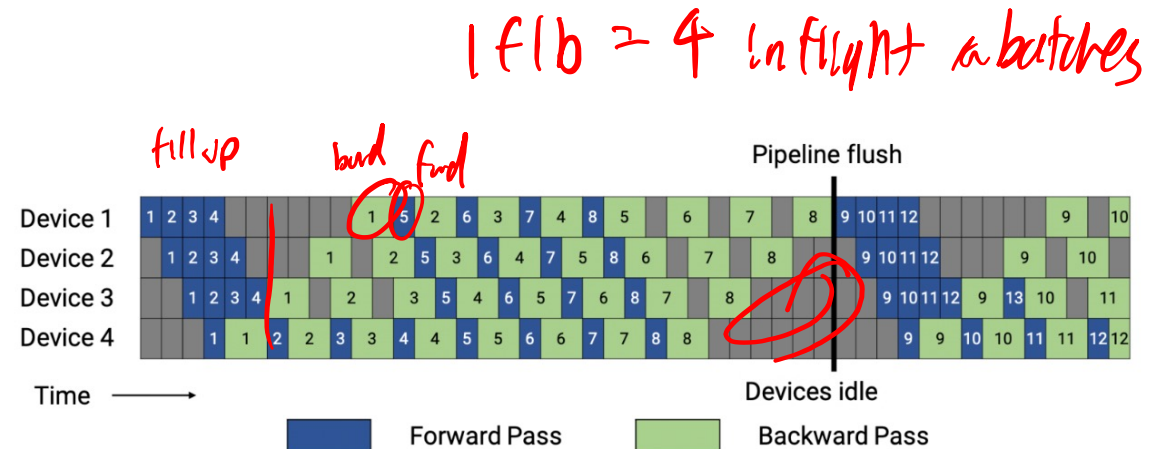
- Need to keep intermediate activations of **all micro-batches** before backprop. How can we improve the pipeline schedule to reduce memory requirements?



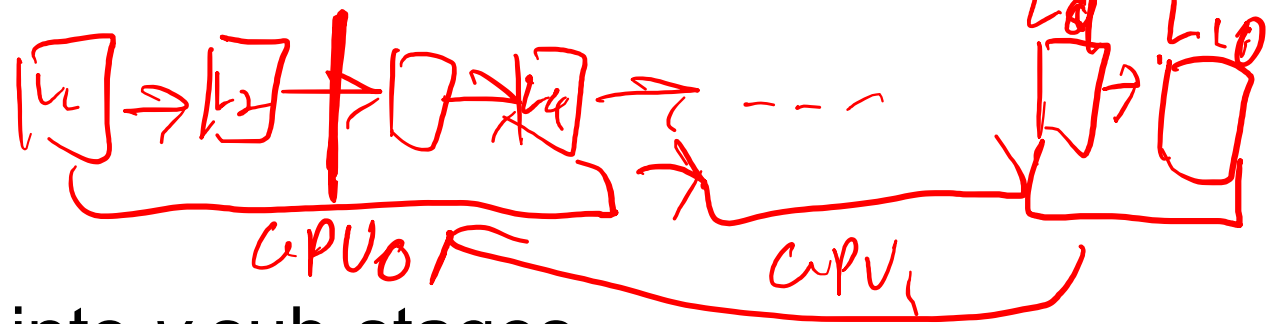
A F A B

1F1B

- One Forward, One Backward in the steady state
- Limits the number of in-flight microbatches to the pipeline depth
- **Reduces memory footprint of pipeline parallelism, but doesn't reduce pipeline bubbles**



Interleaved 1F1B



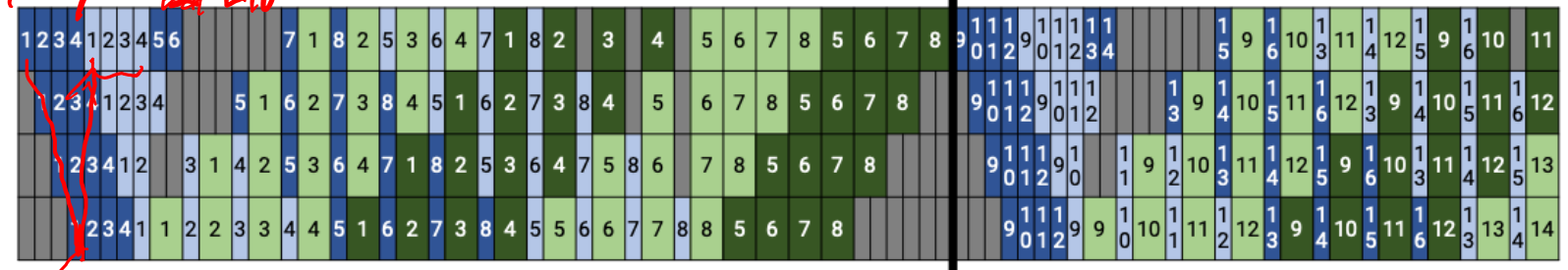
- Idea: Further divide each stage into v sub-stages
 - Before: Device 1 has layers 1-4. After: Device 1 has layers 1-2, 9-10
- Assign each device multiple pipeline stages, each with less computation than before
 - Forward (backward) time of each sub-stage is $\frac{t_f}{v}$ ($\frac{t_b}{v}$)

Costs?

$\sim v$ times
more comm.

$2 \times$

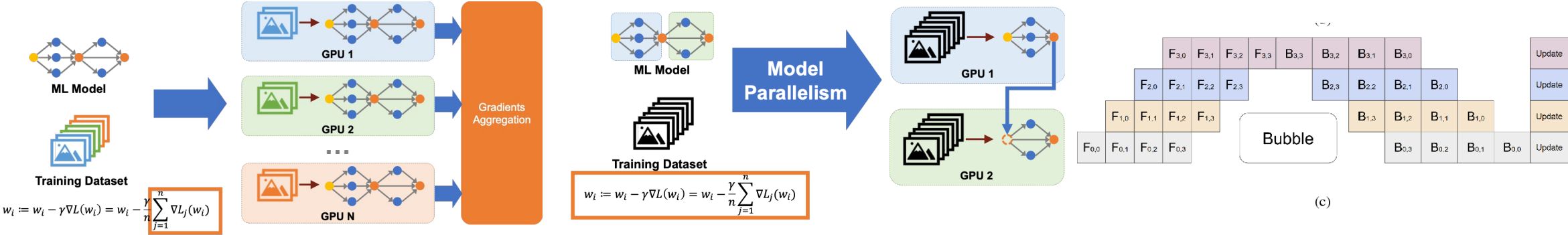
Device 1
Device 2
Device 3
Device 4
Time →



Each device is assigned two chunks. Dark colors show the first chunk and light colors show the second chunk.

$$\text{BubbleFraction} = \frac{(p-1) * \frac{(t_f + t_b)}{v}}{m * t_f + m * t_b} = \frac{1}{v} * \frac{p-1}{m}$$

Summary



	Data Parallelism	Tensor Model Parallelism	Pipeline Model Parallelism
Pros	✓ Massively parallelizable ✓ Require no communication during forward	✓ Support training large models ✓ Efficient for models with large numbers of parameters	✓ Support large-batch training ✓ Efficient for deep models
Cons	❖ Do not work for models that cannot fit on a GPU ❖ Do not scale for models with large numbers of parameters	❖ Limited parallelizability; cannot scale to large numbers of GPUs ❖ Need to transfer intermediate results in forward/backward	❖ Limited utilization: bubbles in forward/backward

Hybrid Parallelism

- Data Parallelism: Higher training throughput if model fits in memory
- Tensor Parallelism: Fit large model into GPU, and there exists high BW interconnect between few GPUs
- Pipeline Parallelism: Fit large model into GPU, and we need to scale to many servers over larger network. However, bubbles limit utilization.

Hybrid Parallelism

Your task: Train a 70B-parameter transformer (80 layers, hidden size 8192) on 512 H100s: 64 servers × 8 GPUs, 80 GB each, NVLink inside a server, 400 Gb/s InfiniBand between servers.

Does it fit? $70 \text{ B} \times 10^9 / \text{param} = 1.1 \text{ TB} \gg 80 \text{ GB}$.

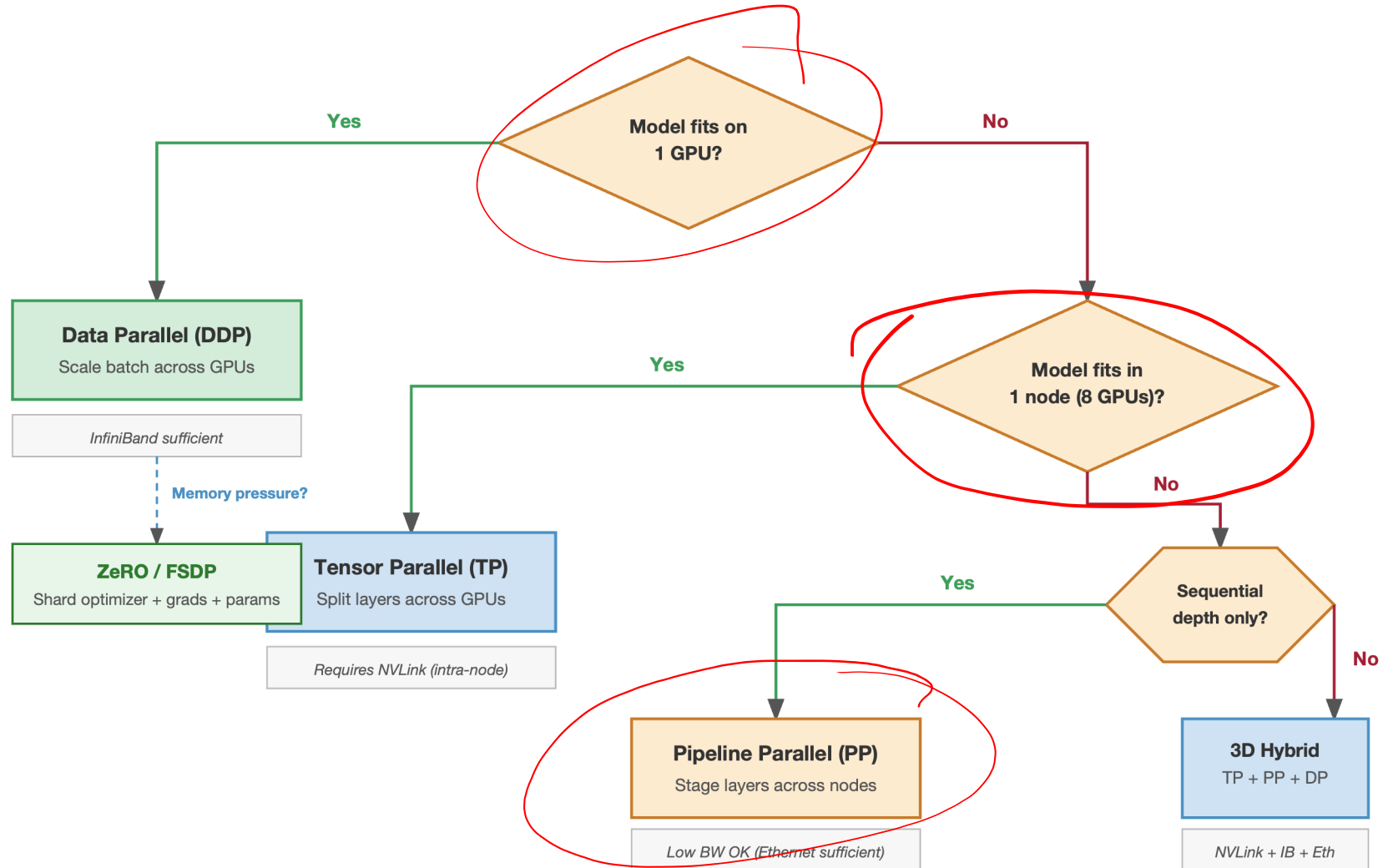
TP x 8 within a node: $1.1 \text{ TB} / 8 = 140 \text{ GB} \gg 80 \text{ GB}$

PP until it fits (across nodes): $PP = 2 \rightarrow \cancel{40 \text{ GB}} 70 \text{ GB} < ? 80$
 $PP = 4$ $\rightarrow 35 \text{ GB} \checkmark$ no $\rightarrow$ b/c activations

32 GPUs in one Pipeline

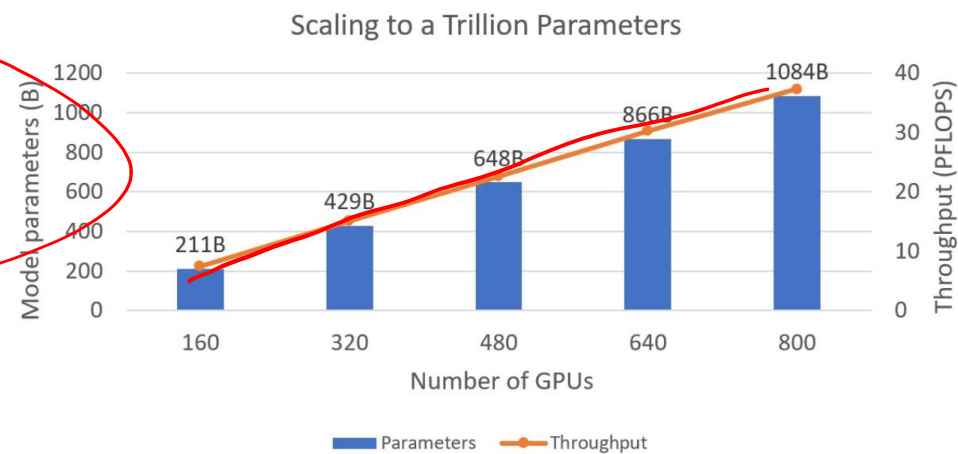
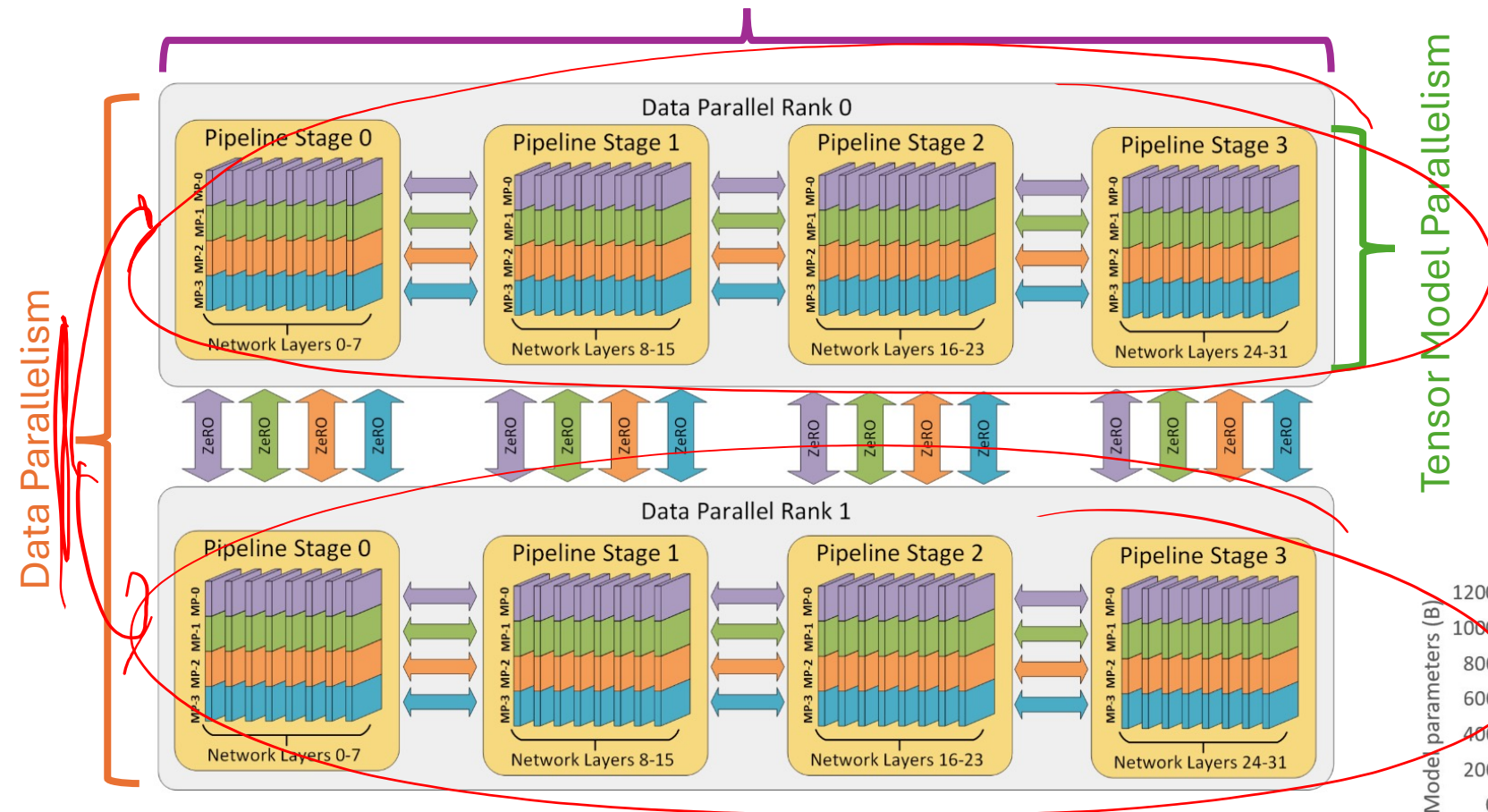
$512 / 32 = 16 \text{ replicas} \rightarrow \text{DP} = 16$

Parallelism Flowchart



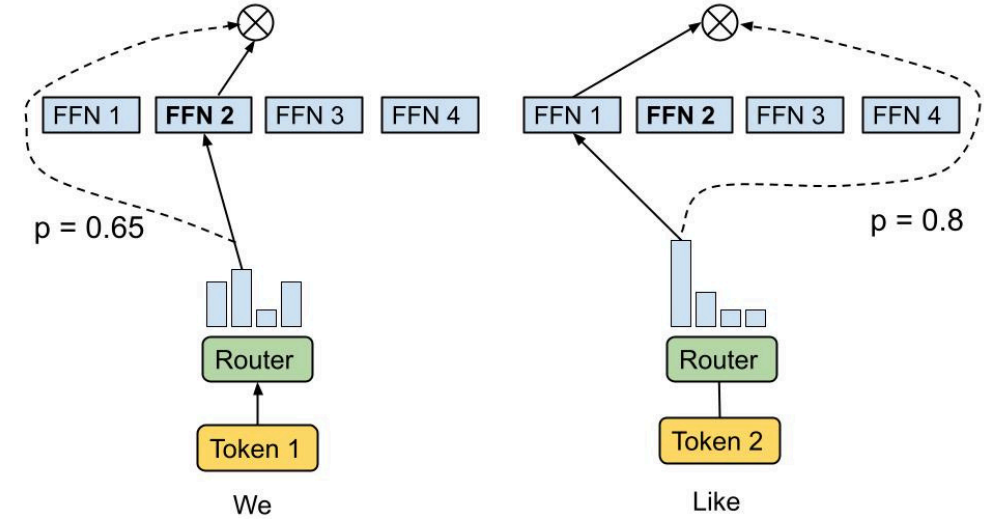
DeepSpeed: 3D Model Parallelism

Pipeline Model Parallelism



Other Flavors of Parallelism

- **Expert Parallelism:** Shard/Replicate Experts (we'll discuss later) across GPUs
- **Sequence Parallelism:** Parallelize Layernorm, Dropout along sequence dimension of transformer
- **Context Parallelism:** Partition entire layer along sequence dimension
- ...



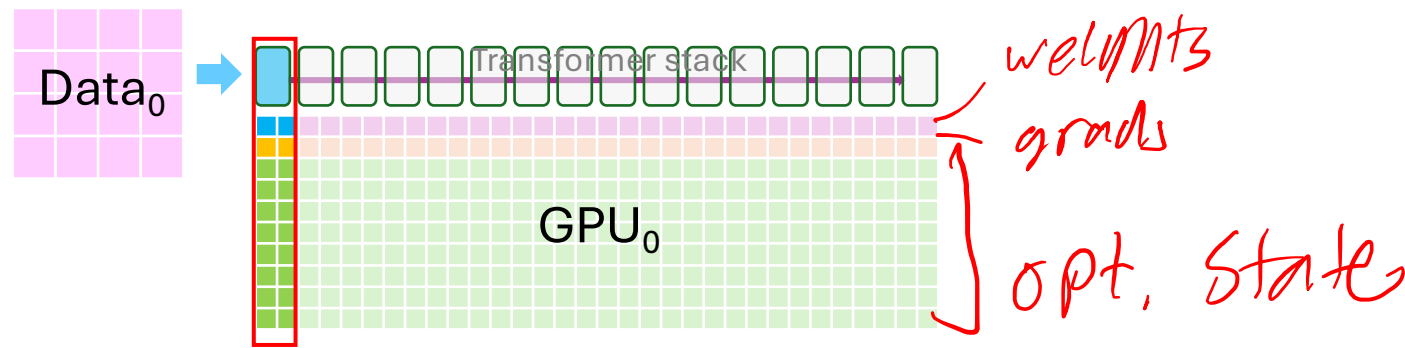
How else can we train larger models?

- Each GPU needs to store entire model!
- Ex: Training a 7B parameter Llama model in mixed precision. Does the full training state fit on an H100-80GB GPU?

2B weights, 2B grads

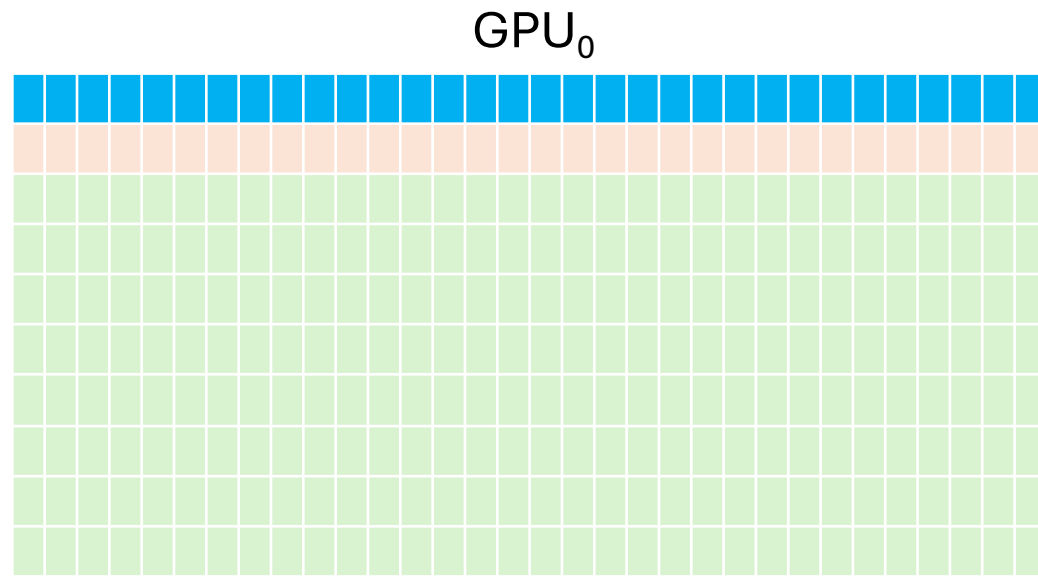
Optimizer state = master weights, moments, variance
FP32

Understanding Training Memory Consumption



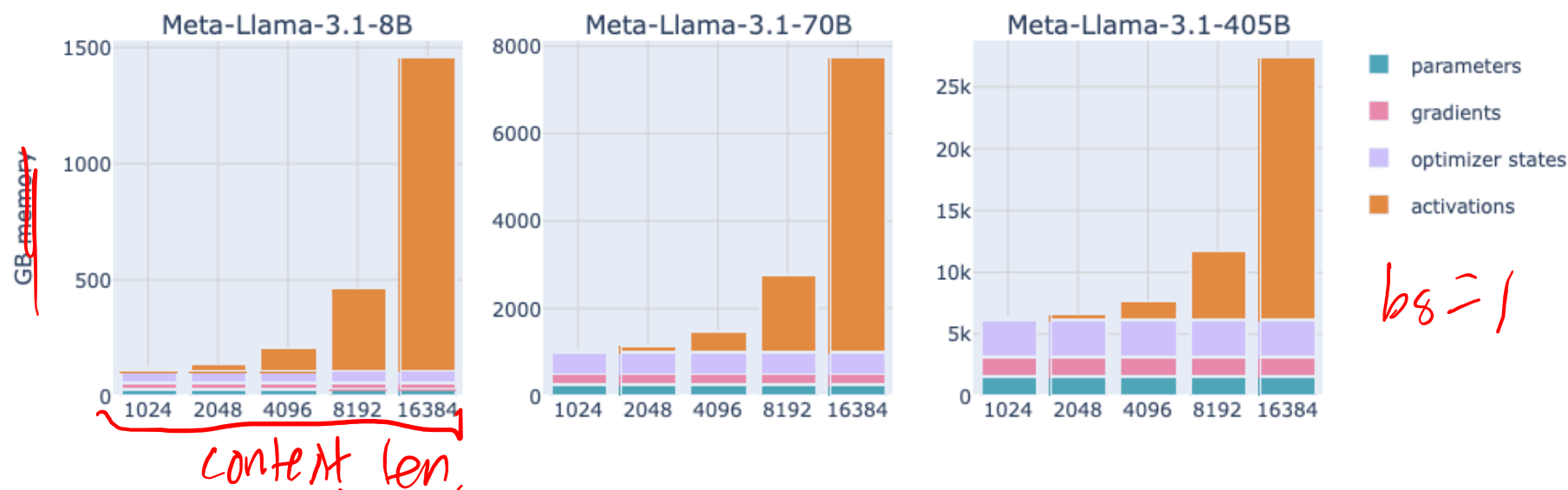
Each cell  represents GPU memory used by its corresponding transformer layer 

Counting Parameters



Not including input batch + activation memory

Activation Memory



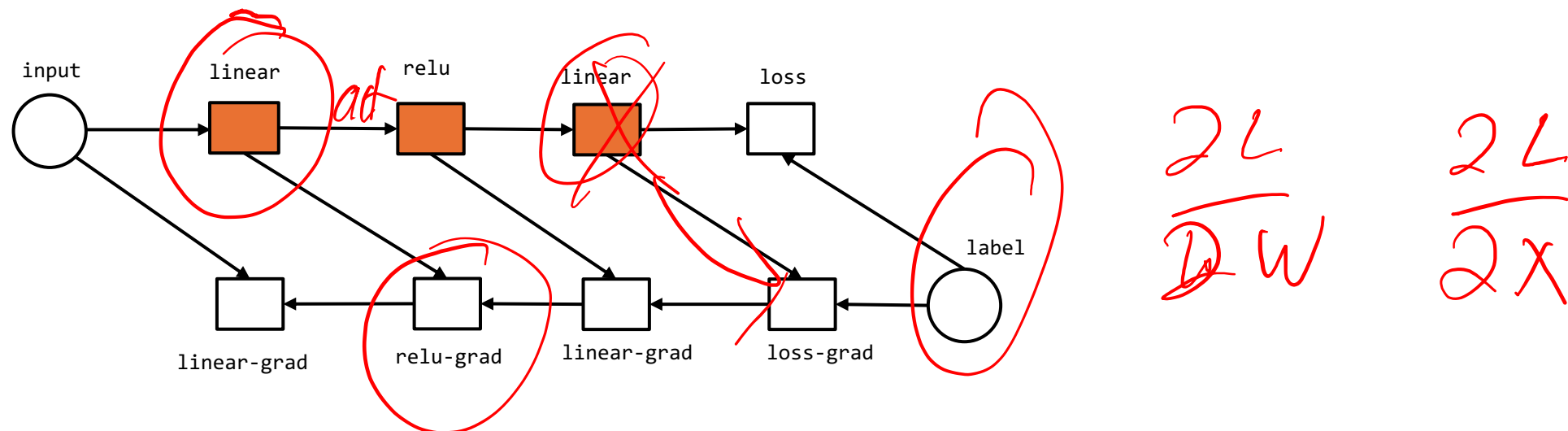
$bs=1$

$$m_{act} = L \cdot \underset{\substack{\uparrow \\ \# \text{ layers}}}{seq} \cdot bs \cdot h \left(34 + \frac{5 \cdot n_{heads} \cdot seq}{L} \right) = O(seq^2)$$

[https://nanotron-ultrascale-playbook.static.hf.space/index.html#memory for activations](https://nanotron-ultrascale-playbook.static.hf.space/index.html#memory%20for%20activations)

<https://arxiv.org/pdf/2205.05198>

Activation Memory



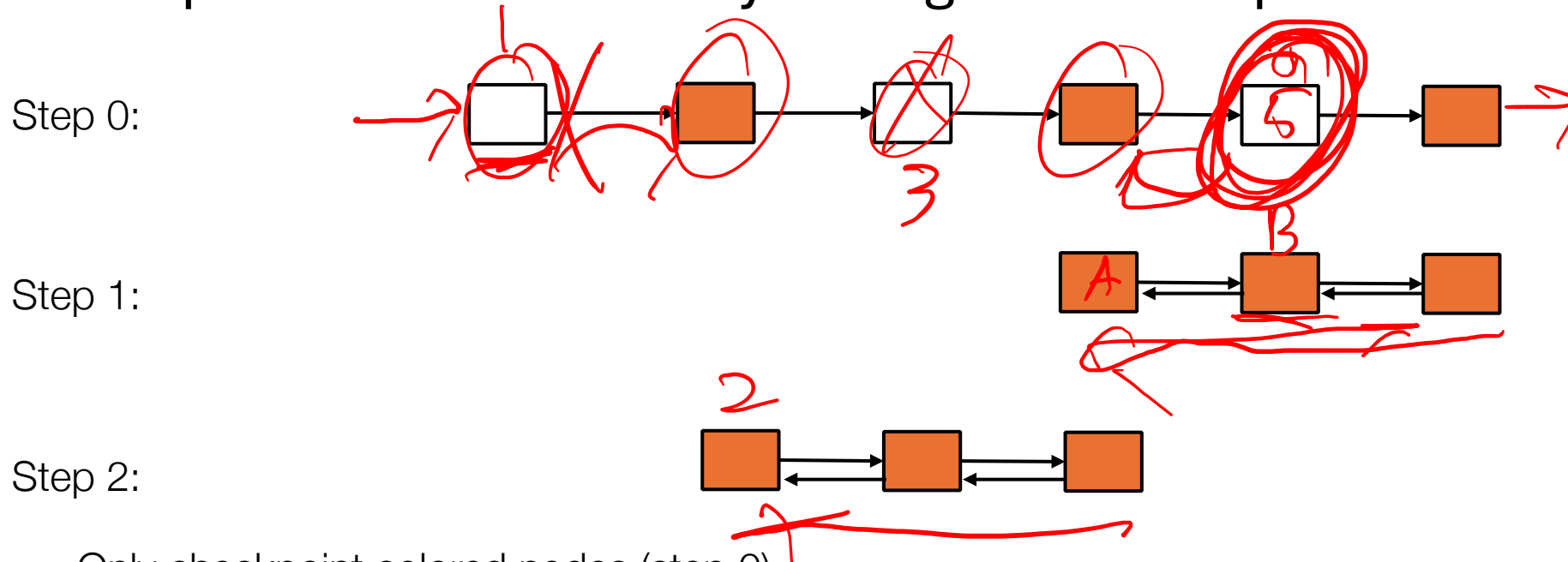
Because the need to keep intermediate value around (checkpoint) for the gradient steps.
Training a N -layer neural network would require $O(N)$ memory.

We will use the following simplified view to combine
gradient and forward computation



Activation Checkpointing

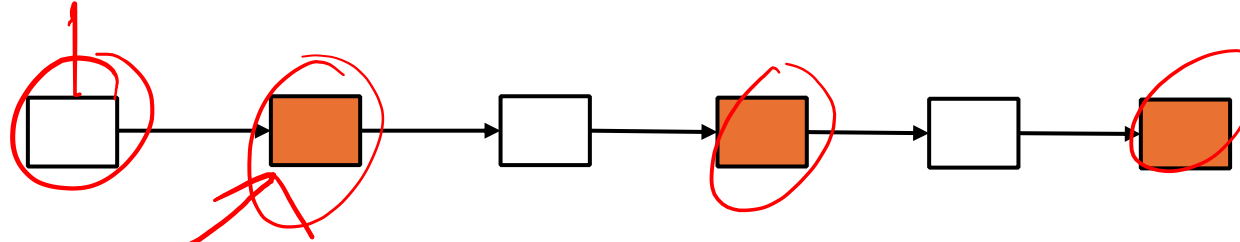
Idea: Discard activations during forward pass to save memory, recompute when necessary during backward pass



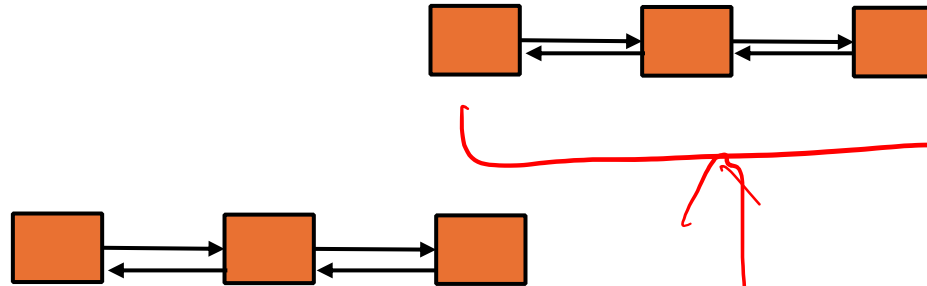
- Only checkpoint colored nodes (step 0)
- Recompute the missing intermediate nodes in small segments (step 1, 2)

Activation Checkpointing

Forward computation



Gradient per segment
with re-computation



For a N layer neural network,
if we checkpoint every K layers, what is the memory cost?

$$\text{mem cost} = O\left(\frac{N}{K}\right) + O(K)$$

Heuristic,
 $\rightarrow K = \sqrt{N}$

ZeRO

- Zero Redundancy Optimizer
- Memory optimization techniques for LLMs
- Deployed via DeepSpeed
<https://www.deepspeed.ai>

ZeRO: Memory Optimizations Toward Training Trillion ~~Parameter Models~~

Samyam Rajbhandari*, Jeff Rasley*, Olatunji Ruwase, Yuxiong He
{samyamr, jerasley, olruwase, yuxhe}@microsoft.com

ZeRO-Offload: Democratizing Billion-Scale Model Training

Jie Ren*, Samyam Rajbhandari[†], Reza Yazdani Aminabadi[†], Olatunji Ruwase[†]
Shuangyan Yang*, Minjia Zhang[†], Dong Li*, Yuxiong He[†]
[†]Microsoft, *University of California, Merced
{jren6, syang127, dli35}@ucmerced.edu, {samyamr, yazdani.reza, olruwase, minjiaz, yuxhe}@microsoft.com

ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning

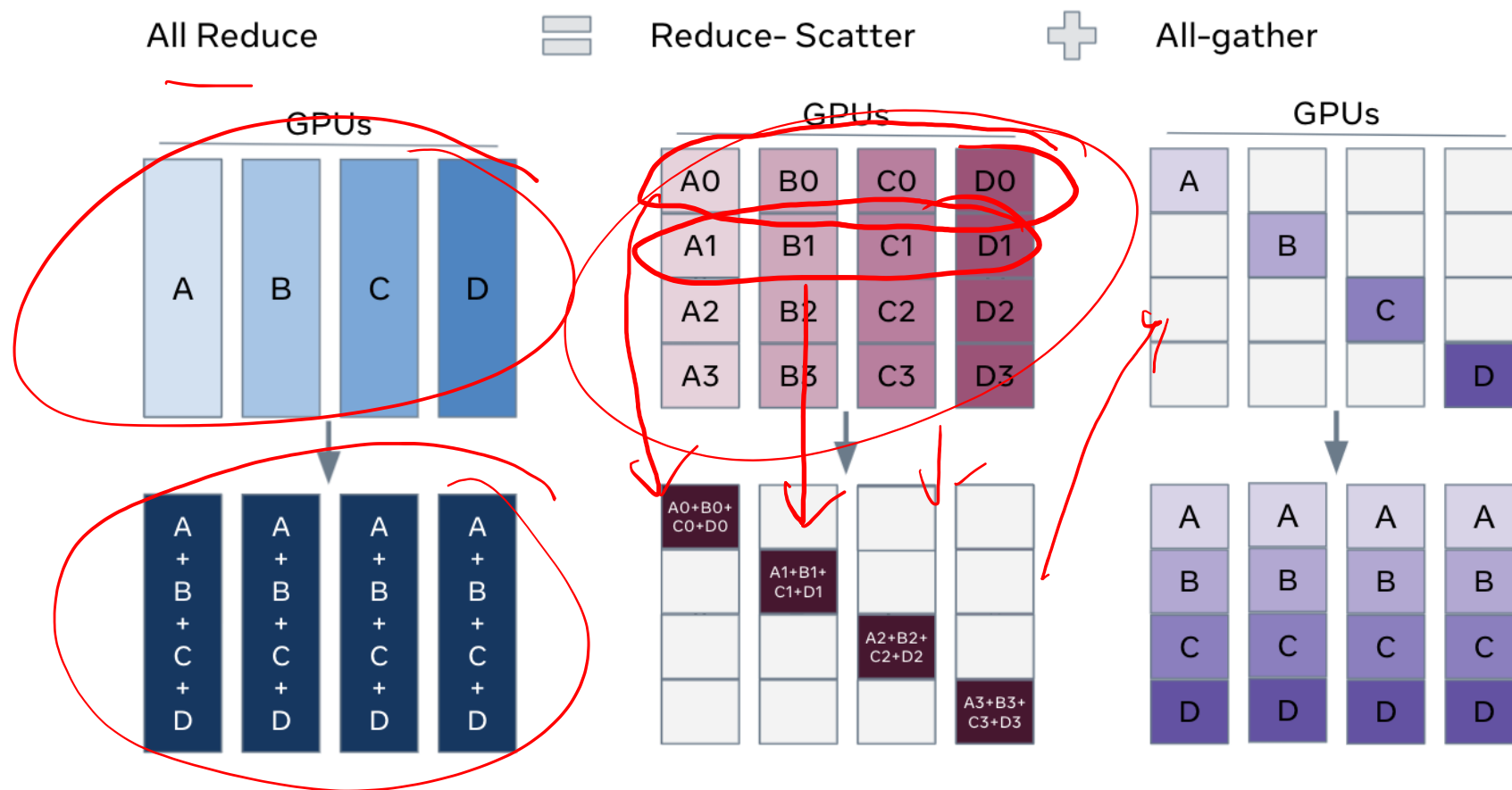
Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, Yuxiong He
{samyamr, olruwase, jerasley, shsmit, yuxhe}@microsoft.com

ZeRO-DP: ZeRO Powered DP

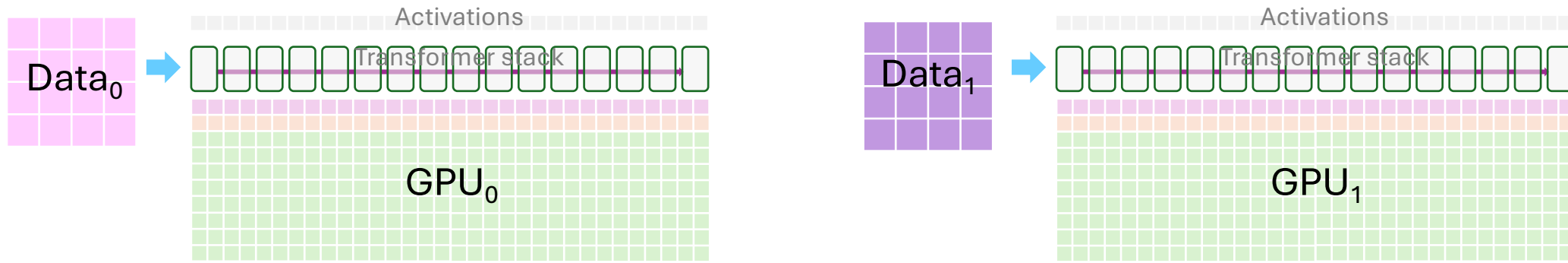
- Remove memory redundancy across data parallel processes
- Stage 1: Partition optimizer state
- Stage 2: Partition gradients
- Stage 3: Partition parameters



ReduceScatter, AllGather

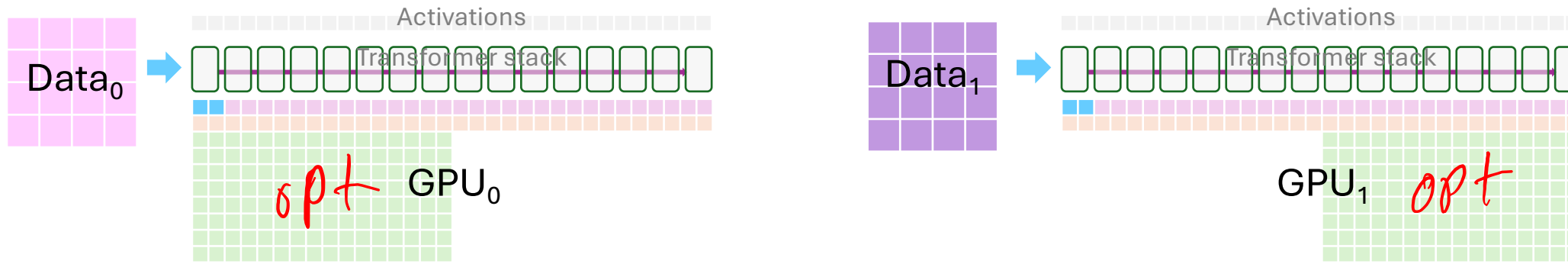


ZeRO Stage 1: Partitioning Optimizer States



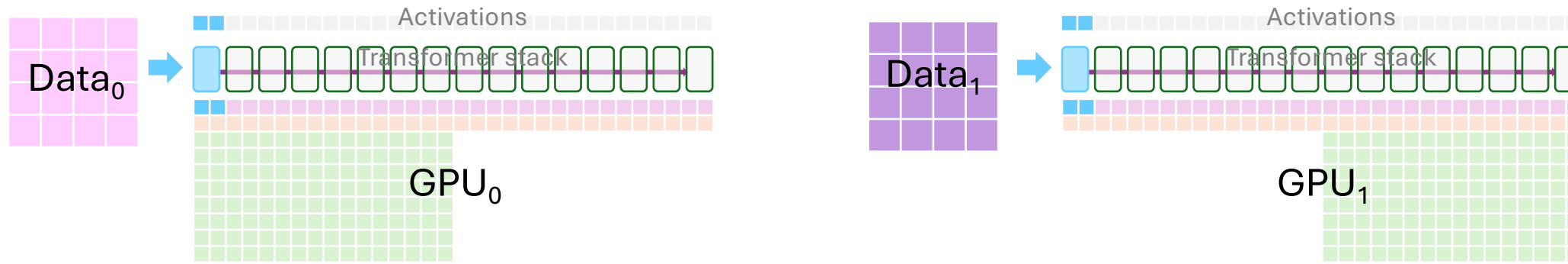
- ZeRO Stage 1

ZeRO Stage 1: Partitioning Optimizer States



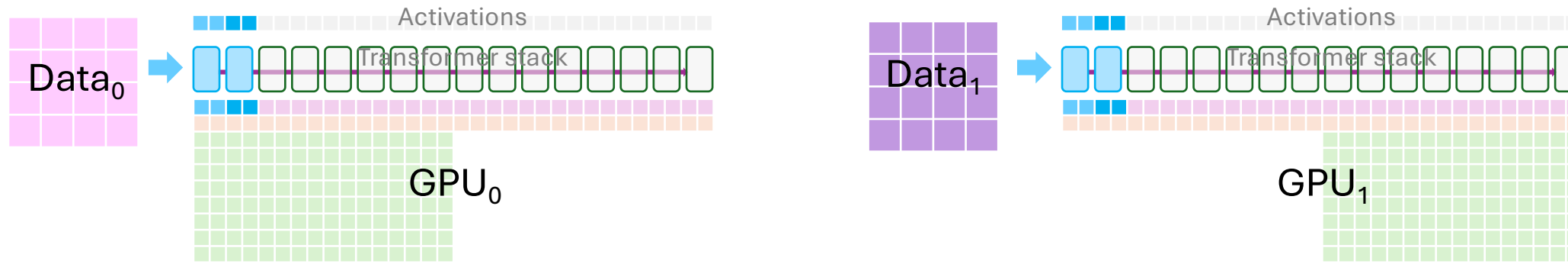
- ZeRO Stage 1
- Partitions optimizer states across GPUs

ZeRO Stage 1: Partitioning Optimizer States



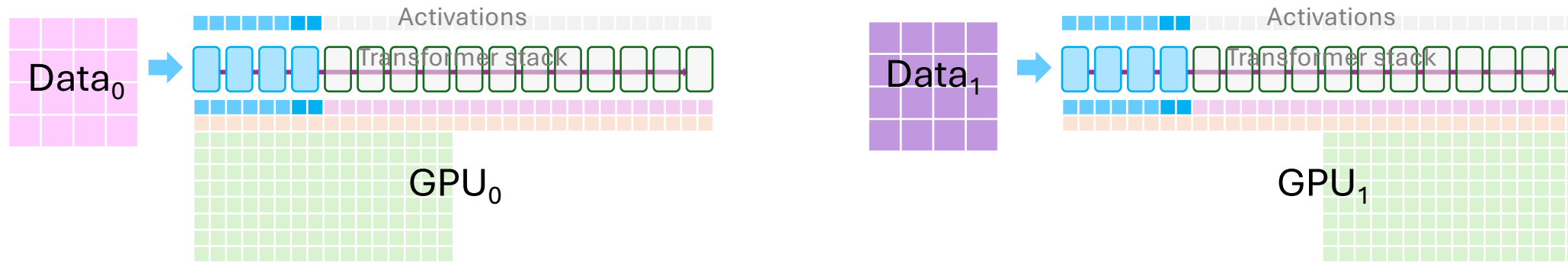
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



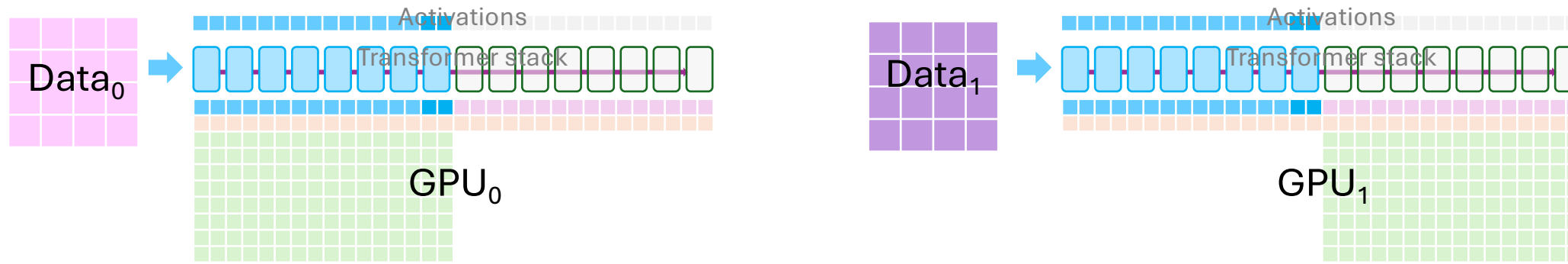
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



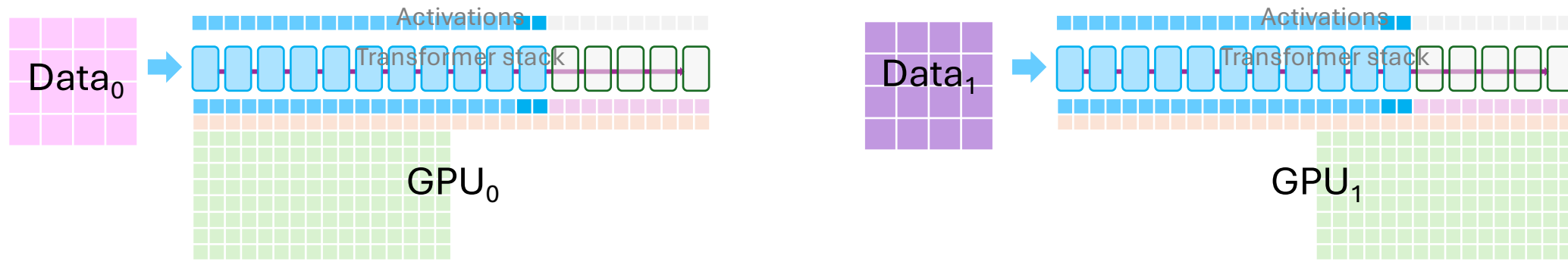
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



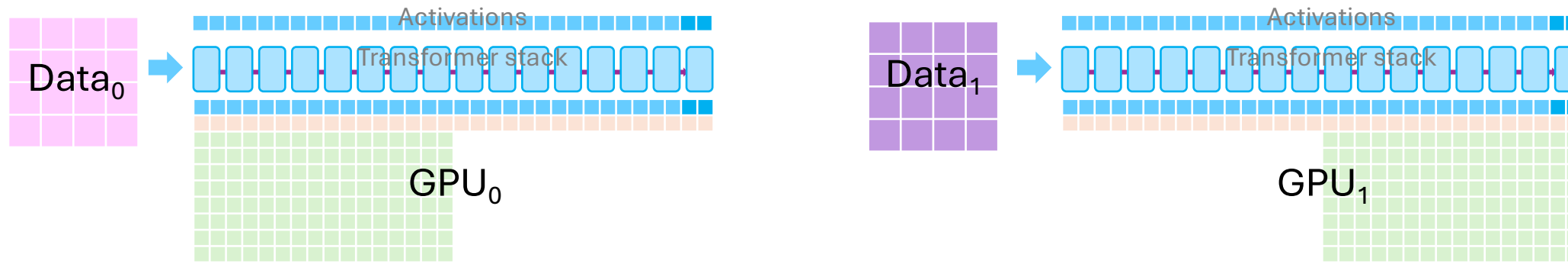
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



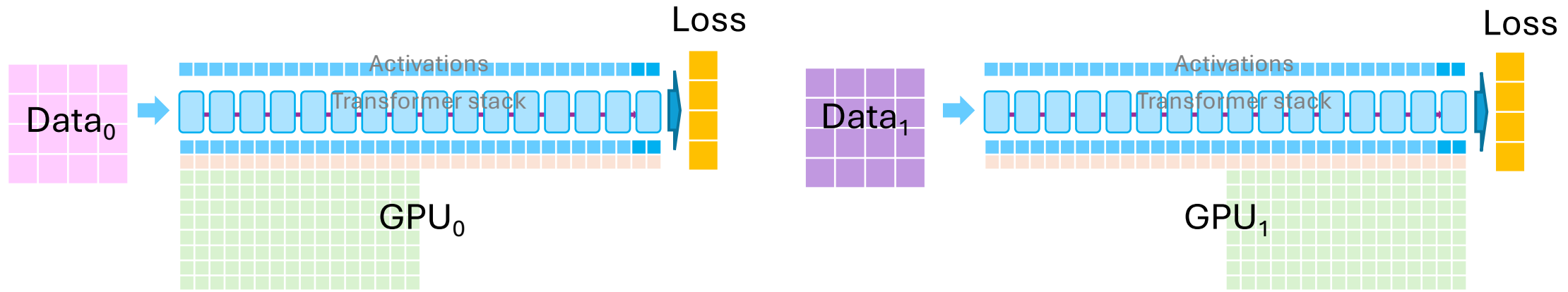
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



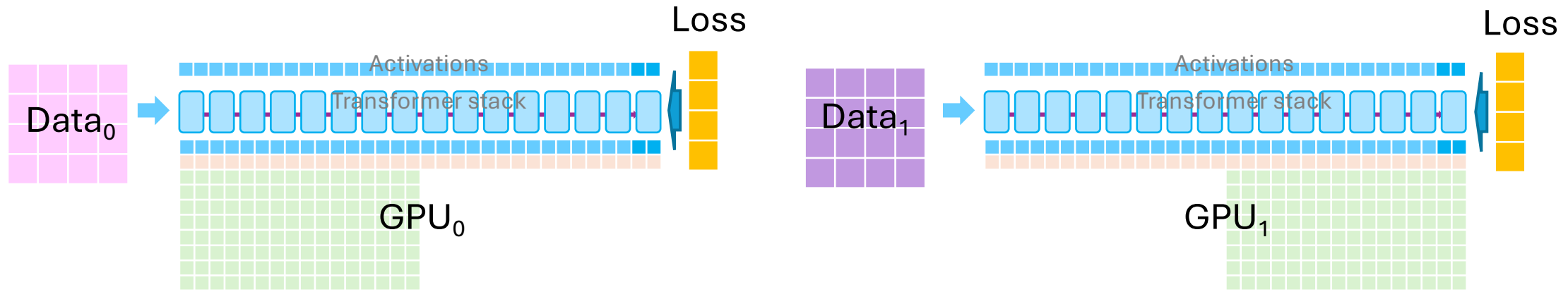
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



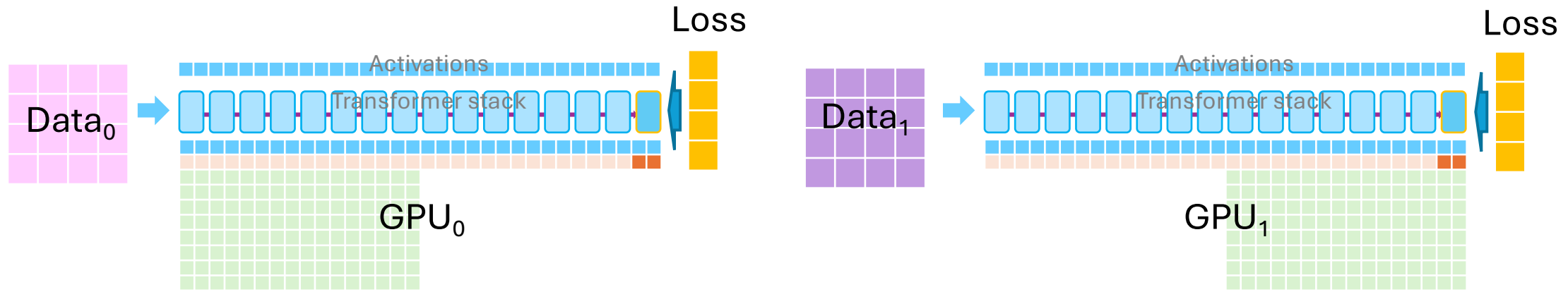
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks

ZeRO Stage 1: Partitioning Optimizer States



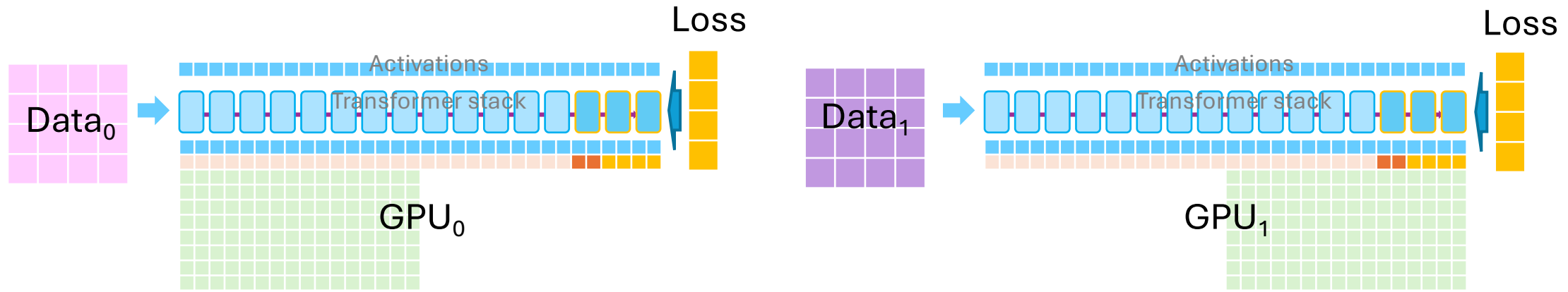
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



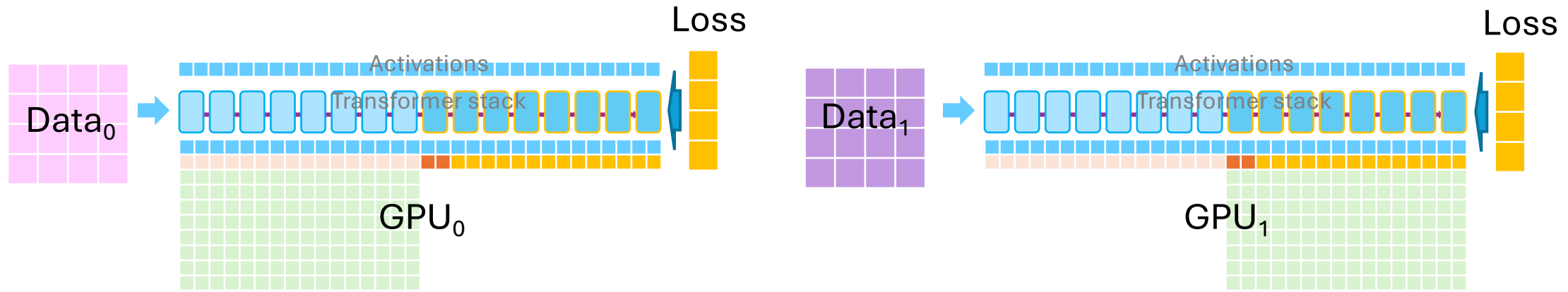
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



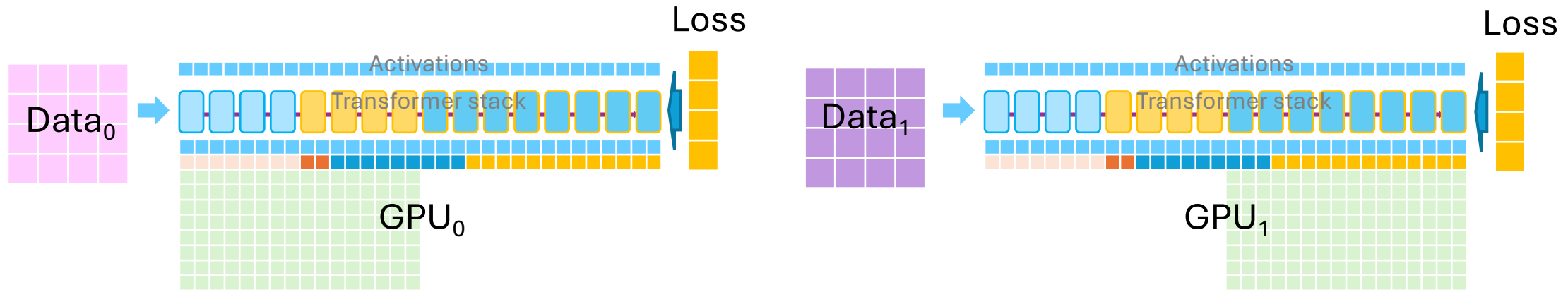
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



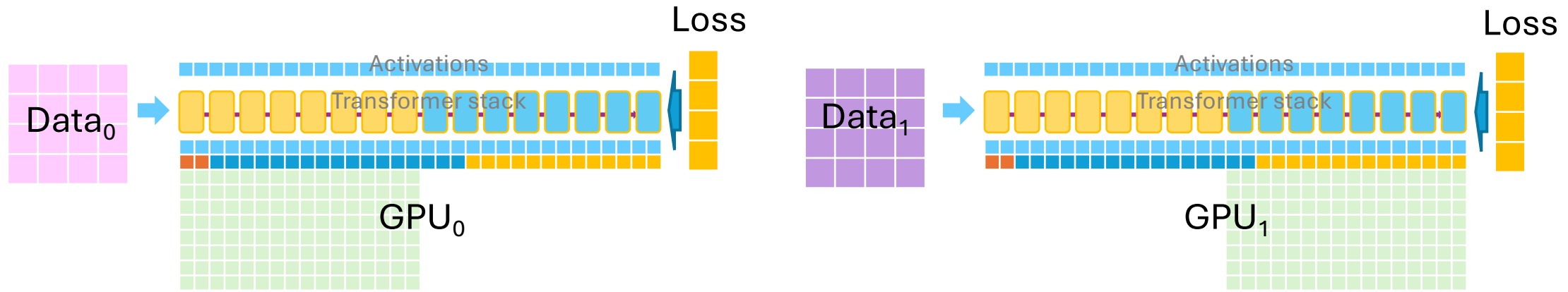
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



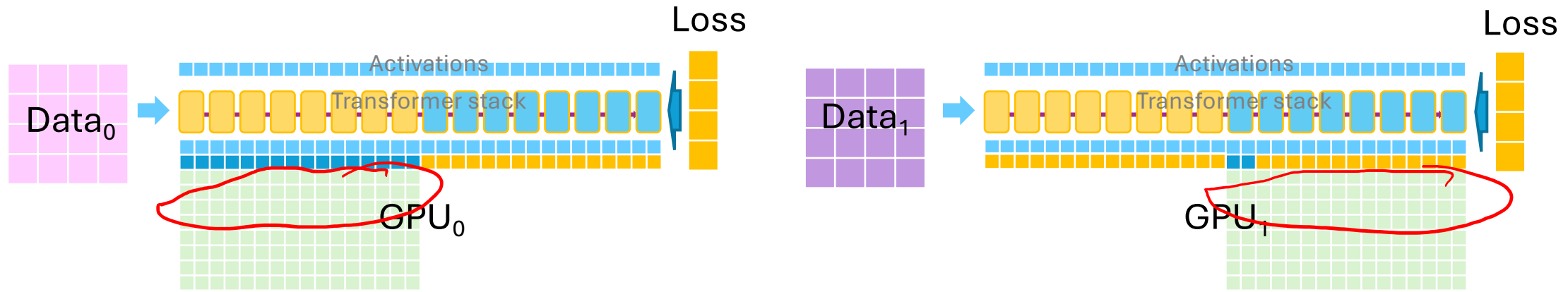
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



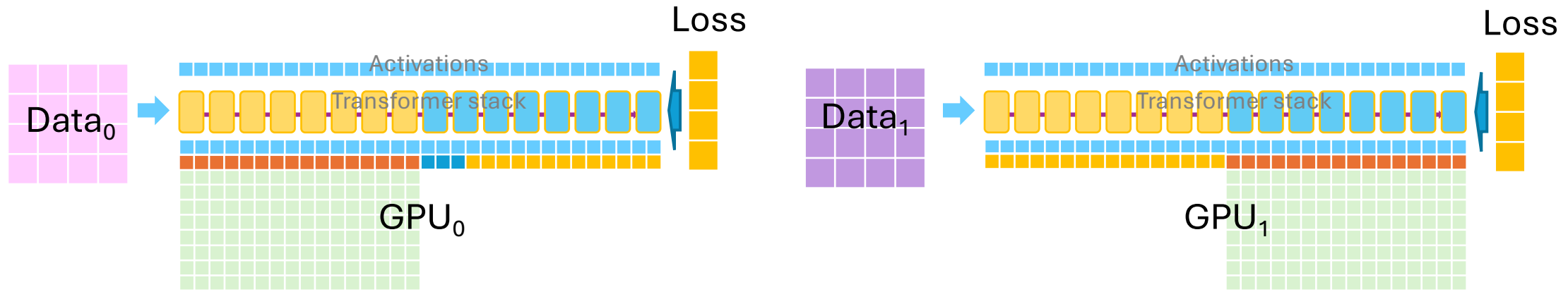
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients

ZeRO Stage 1: Partitioning Optimizer States



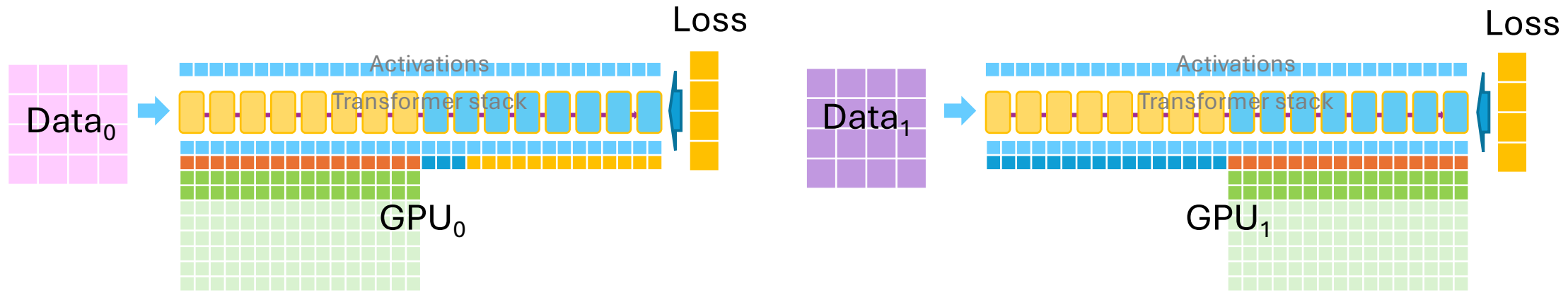
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average

ZeRO Stage 1: Partitioning Optimizer States



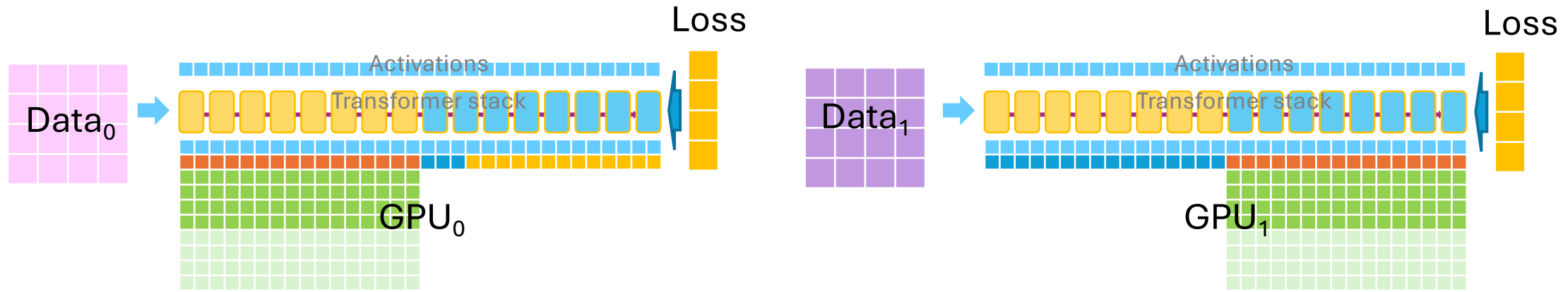
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average

ZeRO Stage 1: Partitioning Optimizer States



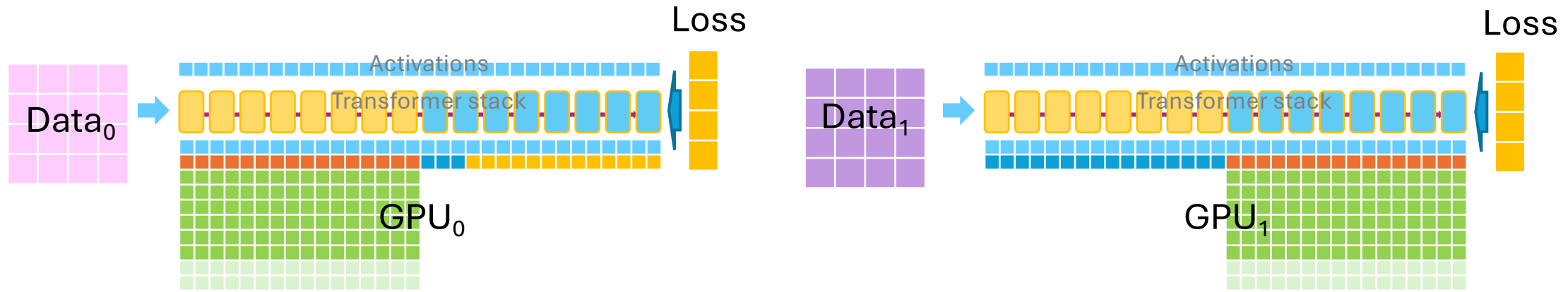
- ZeRO Stage 1
- Partitions optimizer states across GPUs
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer

ZeRO Stage 1: Partitioning Optimizer States



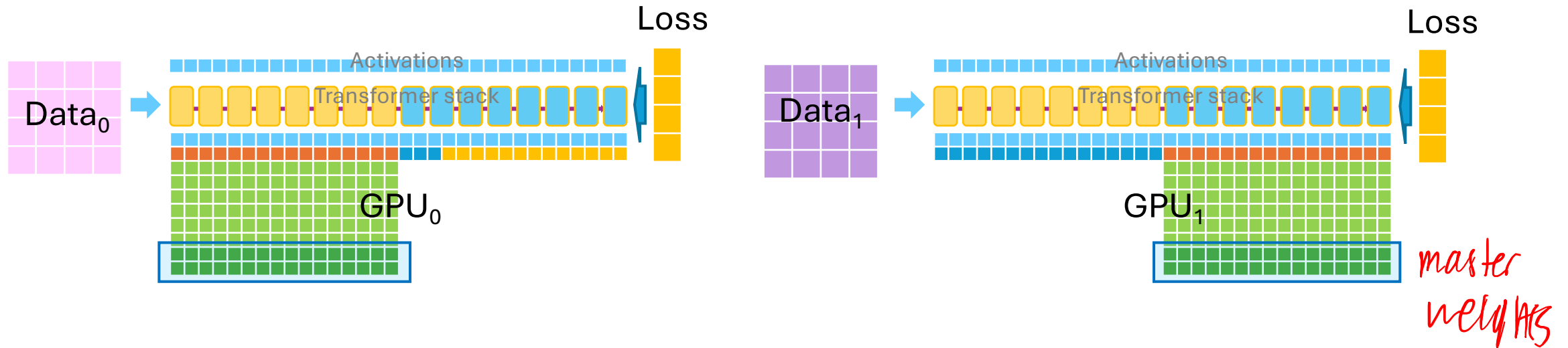
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer

ZeRO Stage 1: Partitioning Optimizer States



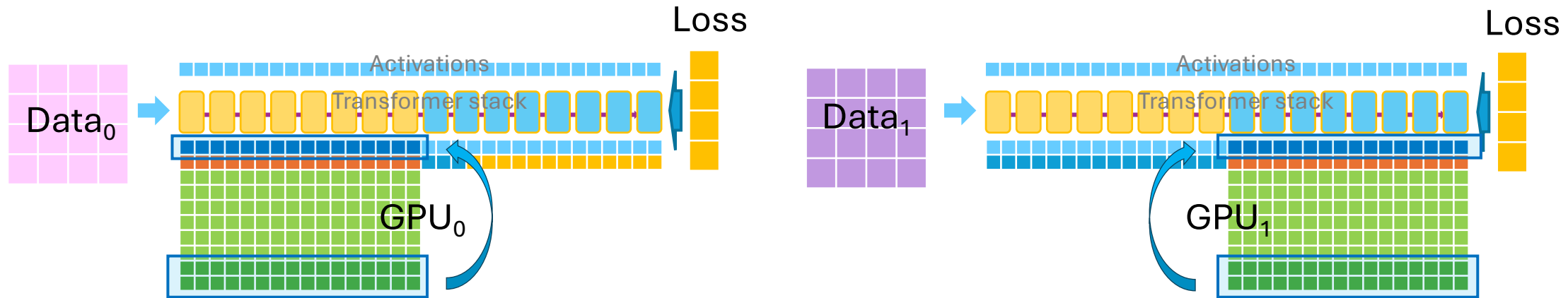
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer

ZeRO Stage 1: Partitioning Optimizer States



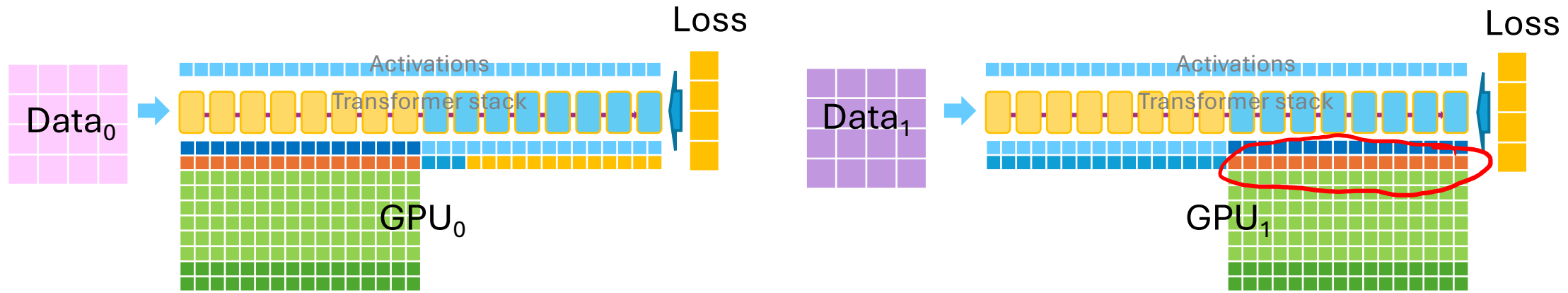
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer

ZeRO Stage 1: Partitioning Optimizer States



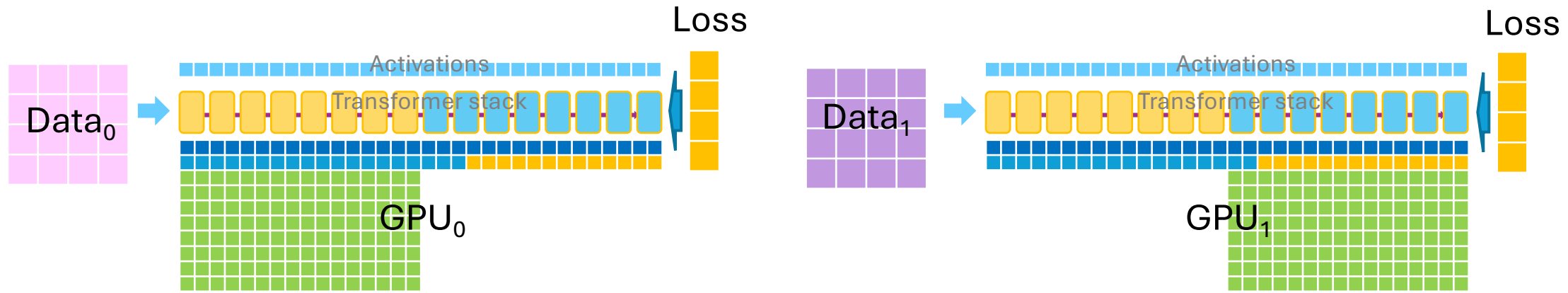
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer
- Update the FP16 weights

ZeRO Stage 1: Partitioning Optimizer States



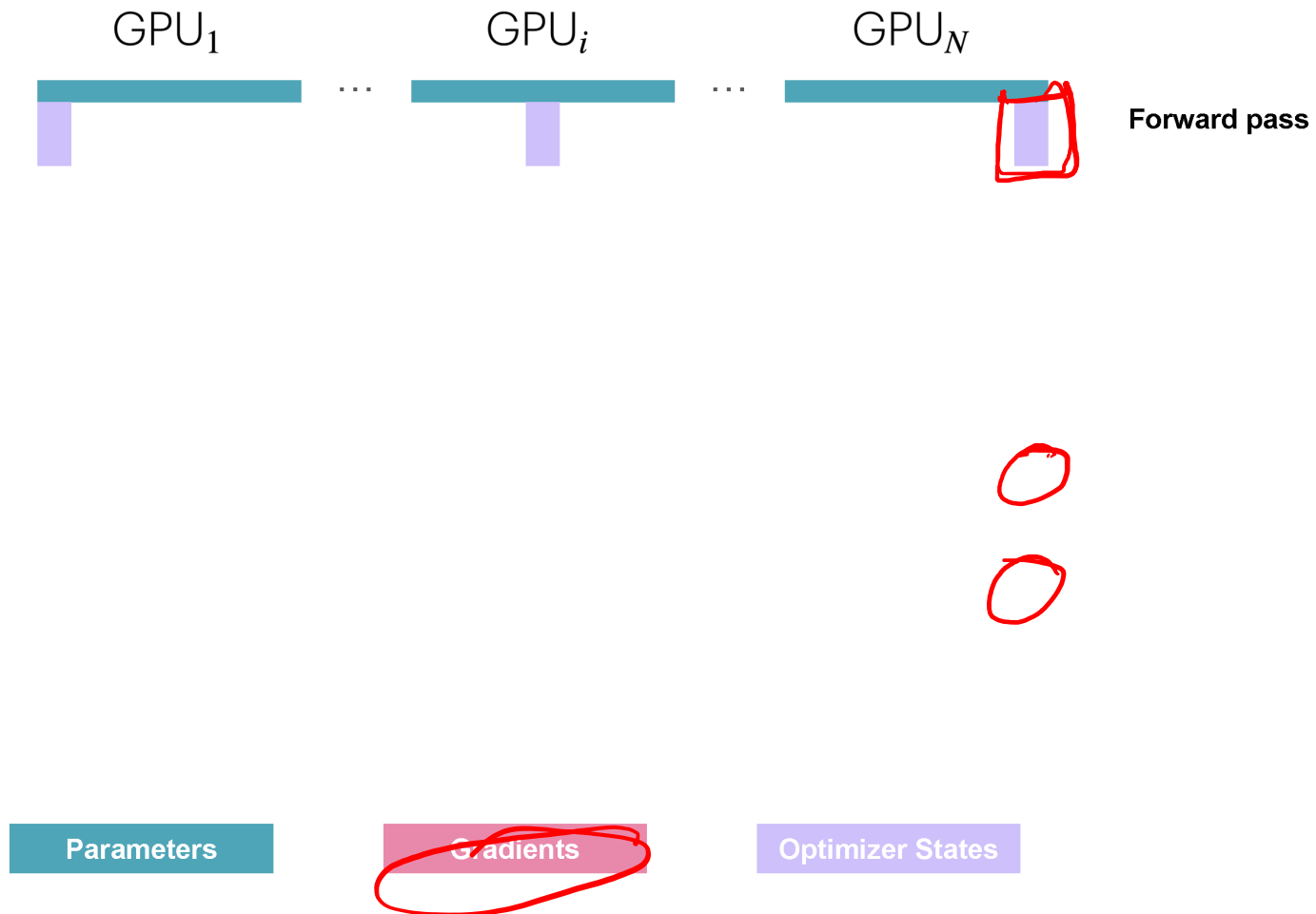
- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer
- Update the FP16 weights
- All Gather the FP16 weights to complete the iteration

ZeRO Stage 1: Partitioning Optimizer States

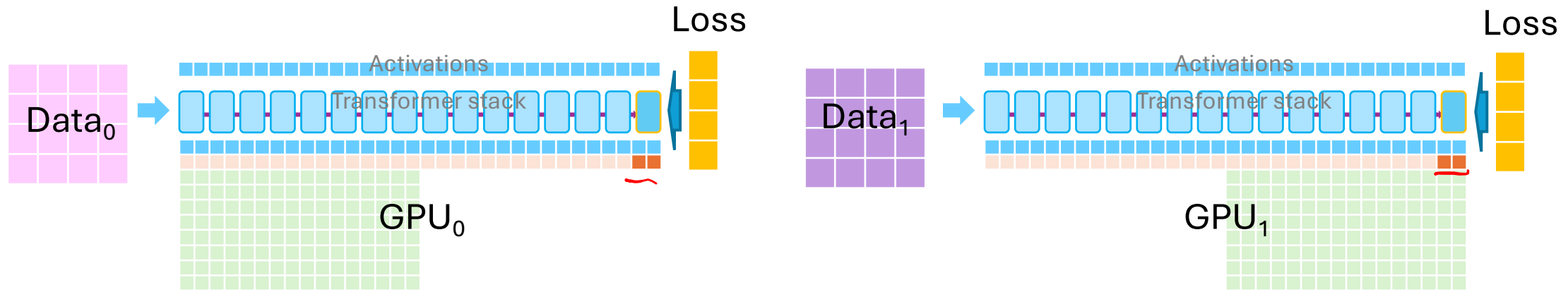


- Run Forward across the transformer blocks
- Backward propagation to generate FP16 gradients and ReduceScatter to average
- Update the FP32 weights with ADAM optimizer
- Update the FP16 weights
- All Gather the FP16 weights to complete the iteration

ZeRO Stage 1: Partitioning Optimizer States

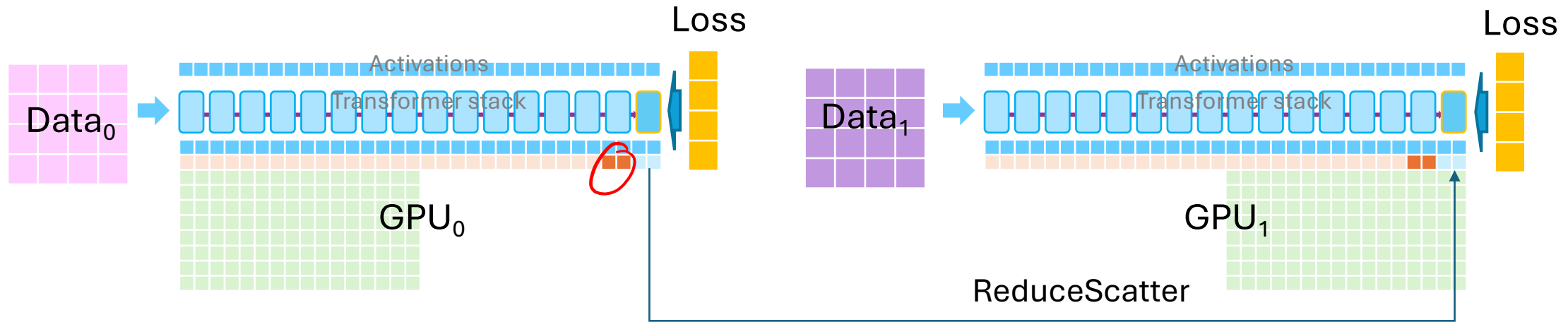


ZeRO Stage 2: Partitioning Gradients



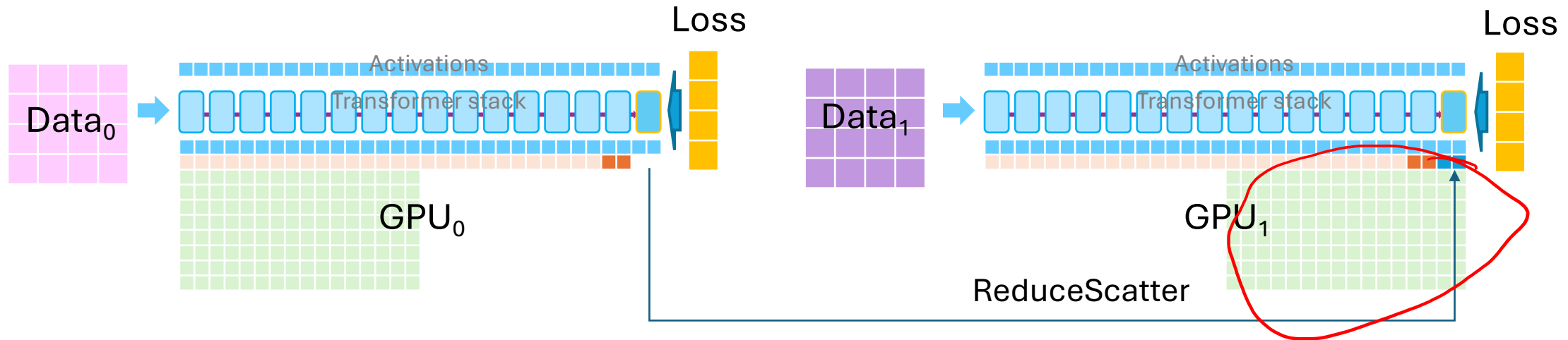
- Partitioning gradients across GPUs
- The forward process remains the same as stage 1

ZeRO Stage 2: Partitioning Gradients



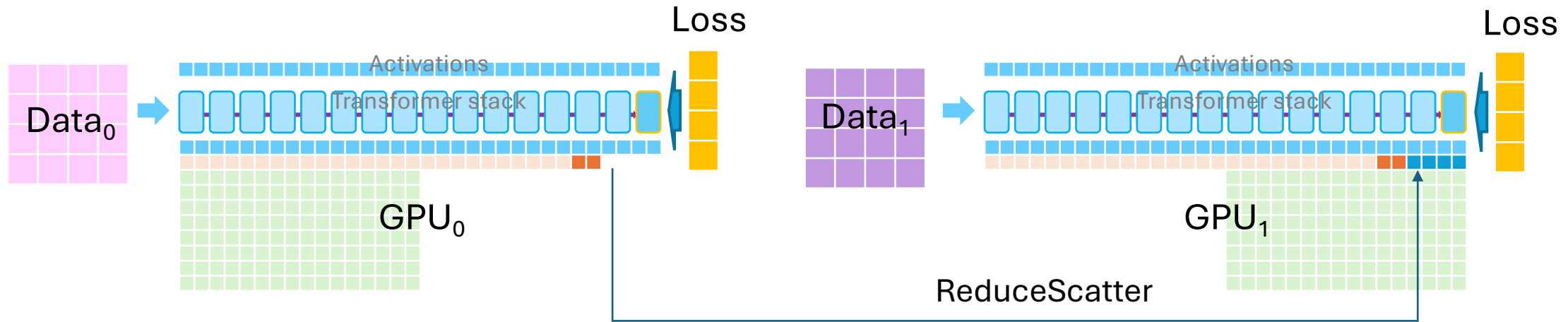
- Partitioning gradients across GPUs
- Perform ReduceScatter right after back propagation of each layer

ZeRO Stage 2: Partitioning Gradients



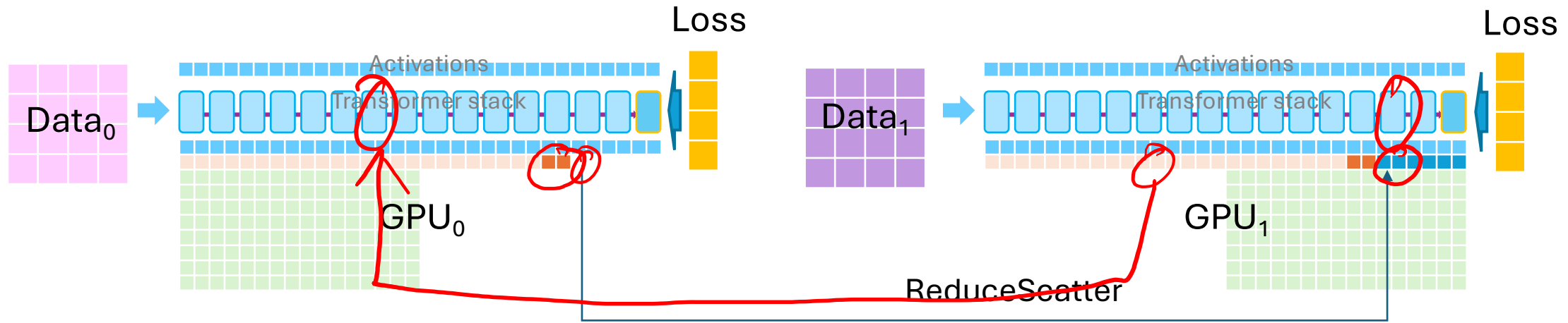
- Partitioning gradients across GPUs
- Only one GPU keeps the gradients after ReduceScatter

ZeRO Stage 2: Partitioning Gradients



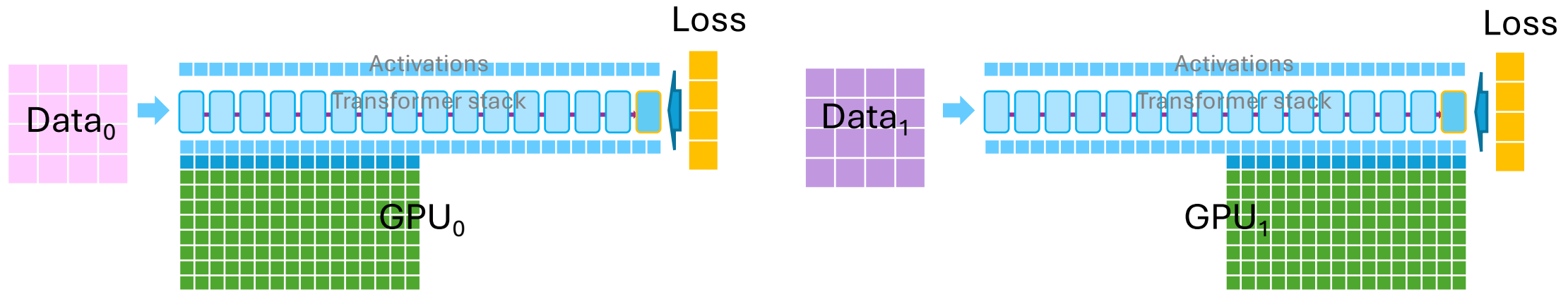
- Partitioning gradients across GPUs
- Reduce gradients on GPUs responsible for updating parameters

ZeRO Stage 2: Partitioning Gradients



- Partitioning gradients across GPUs
- Reduce gradients on GPUs responsible for updating parameters

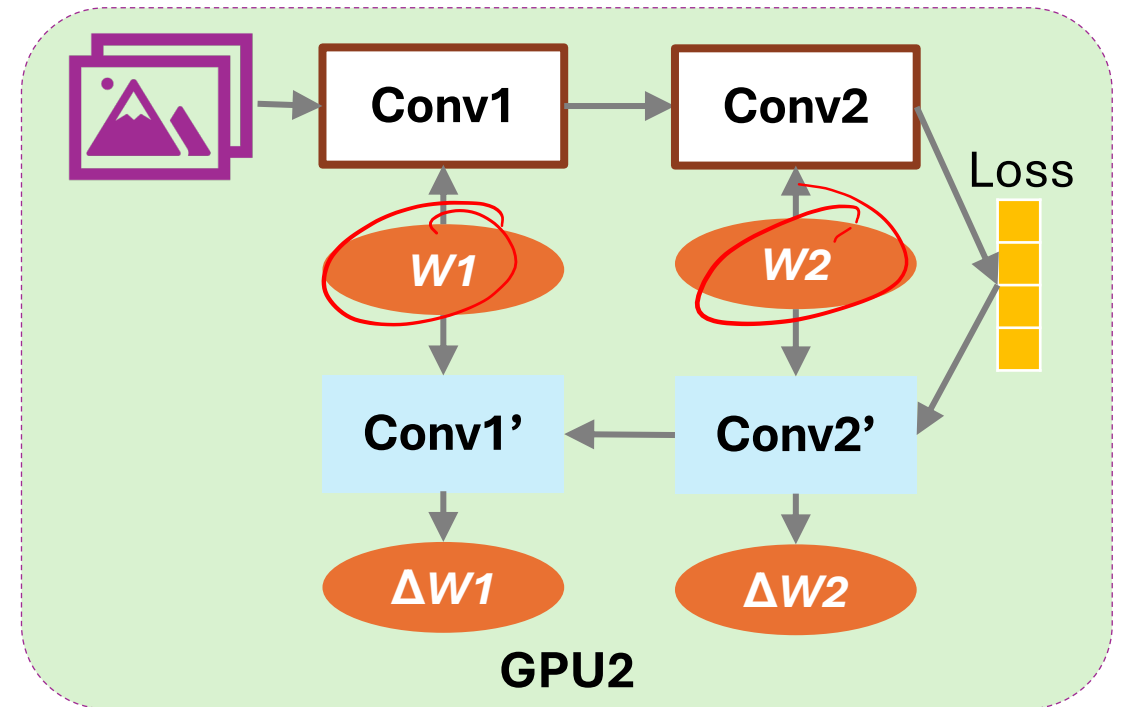
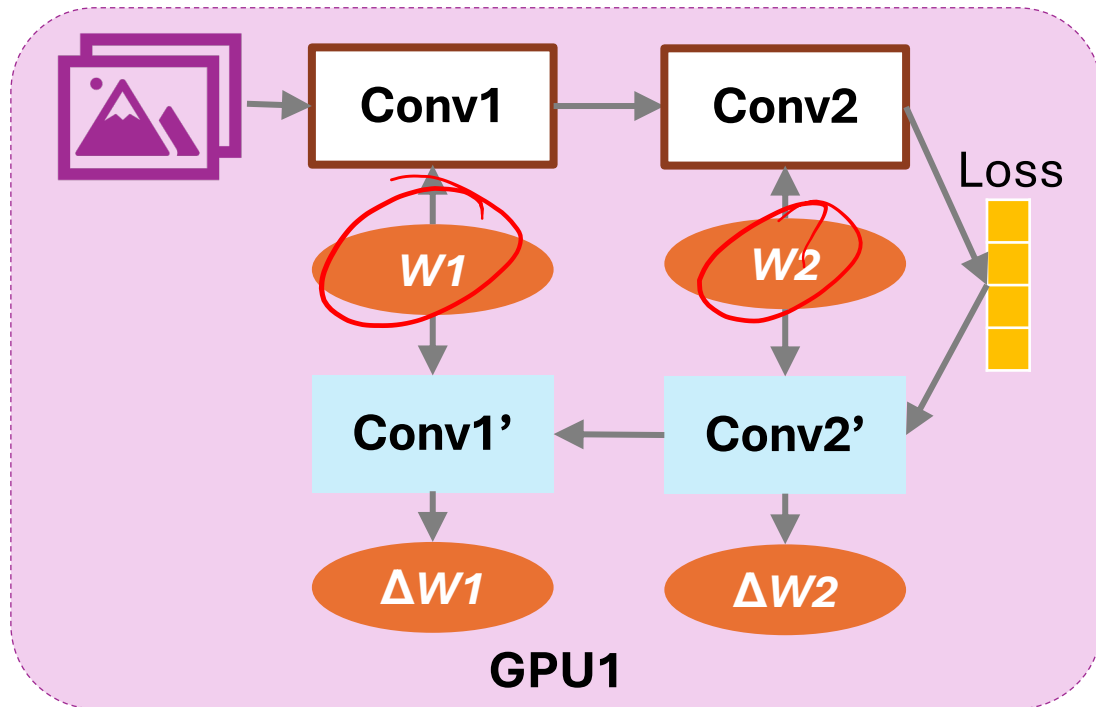
ZeRO Stage 2: Partitioning Gradients



- Partitioning gradients across GPUs
- Reduce gradients on GPUs responsible for updating parameters

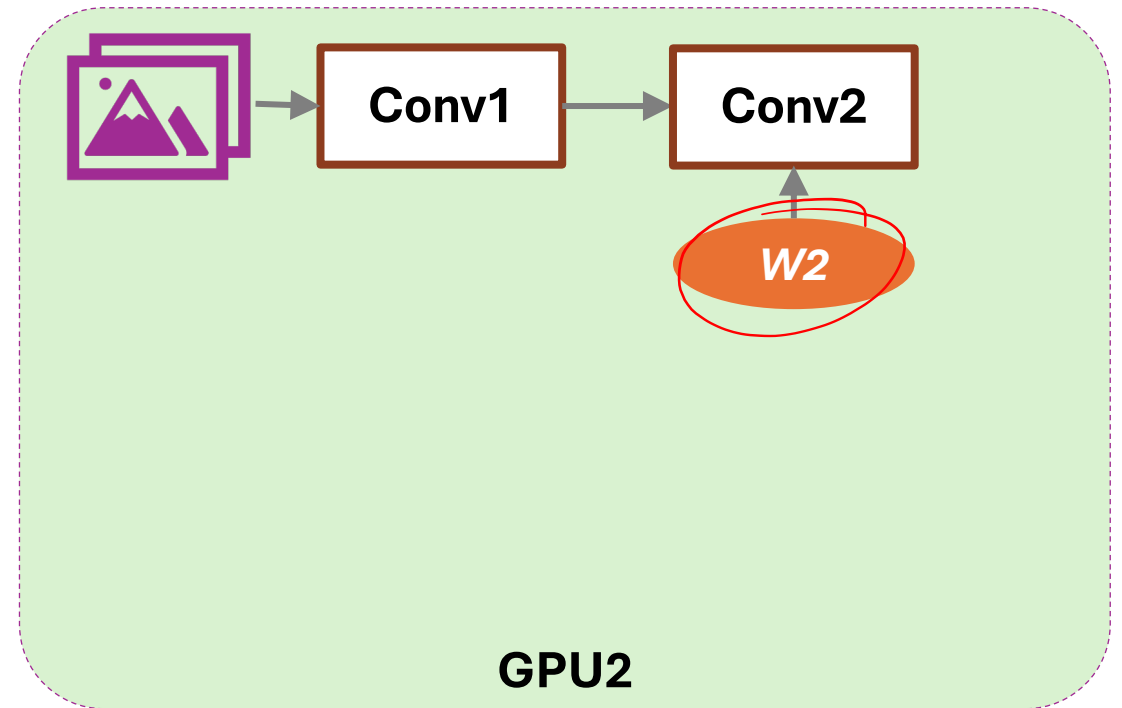
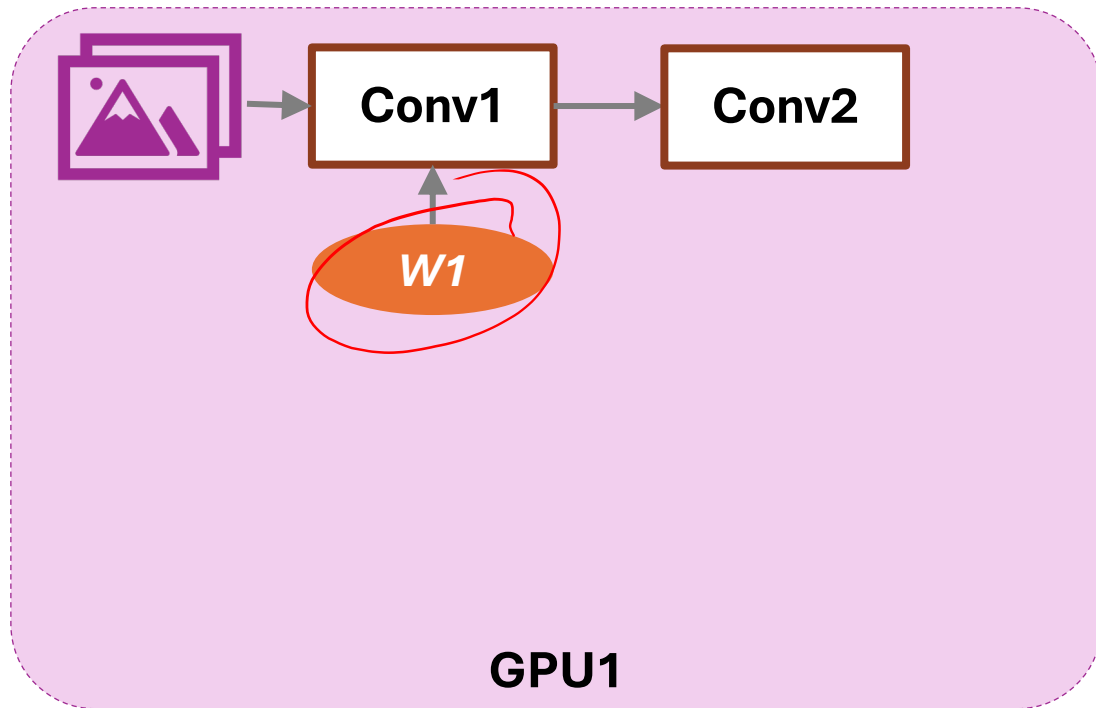
ZeRO Stage 3: Partitioning Parameters

- In data parallel training, all GPUs keep all parameters during training



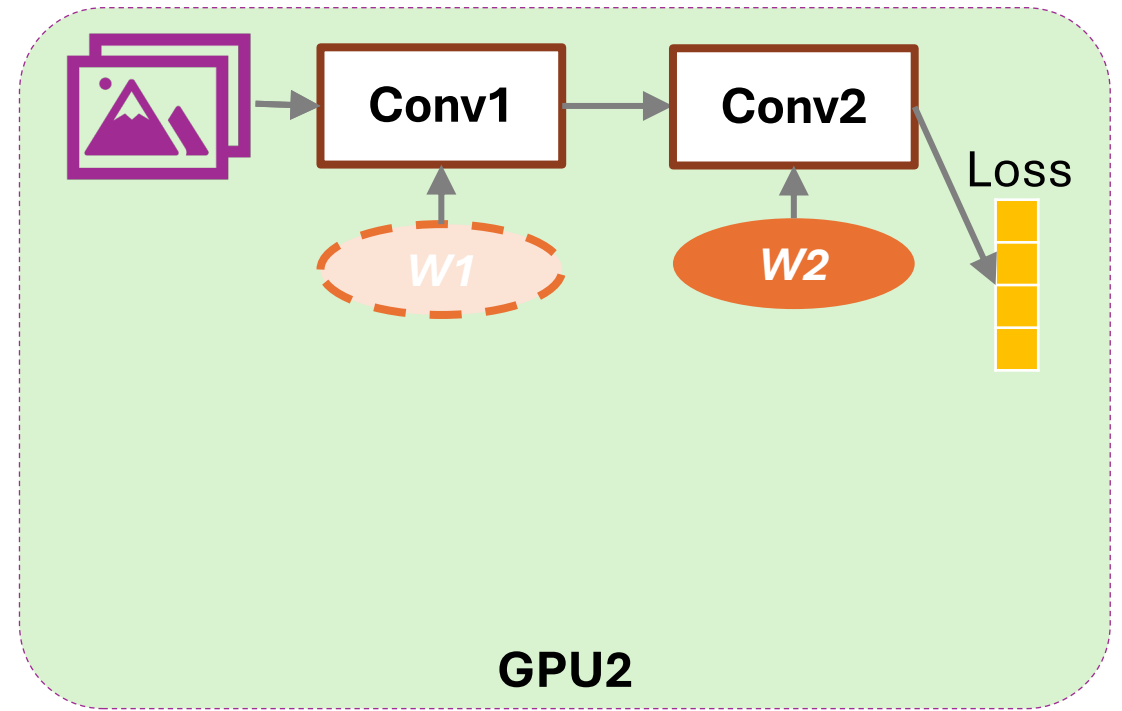
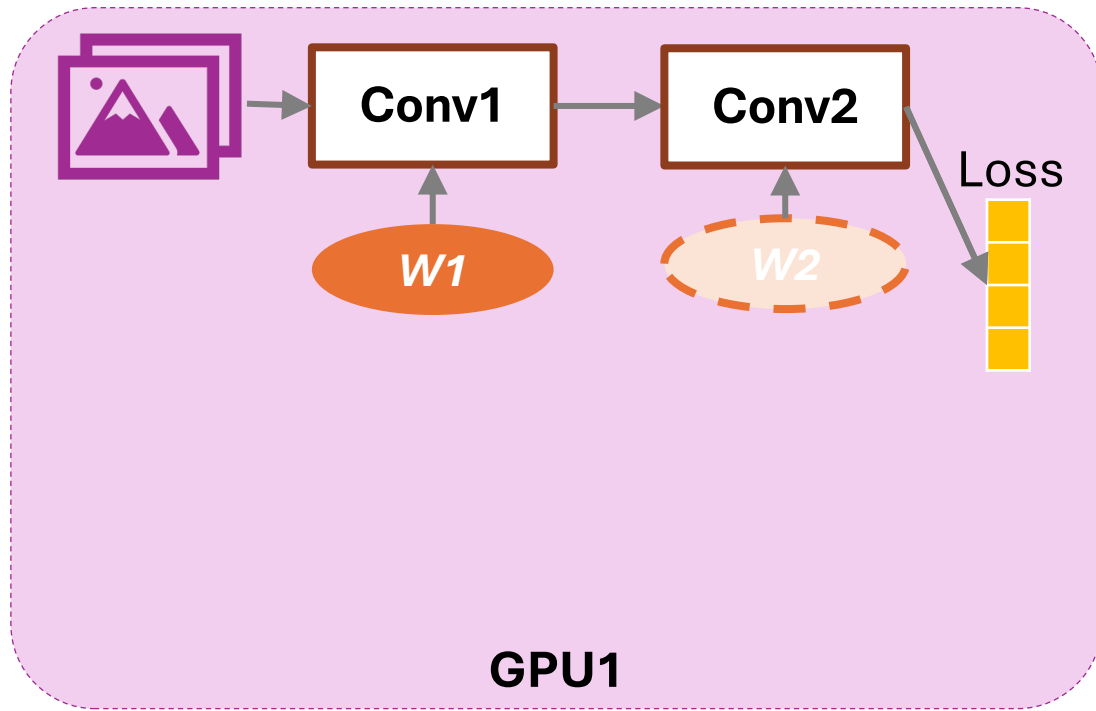
ZeRO Stage 3: Partitioning Parameters

- In ZeRO, model parameters are partitioned across GPUs



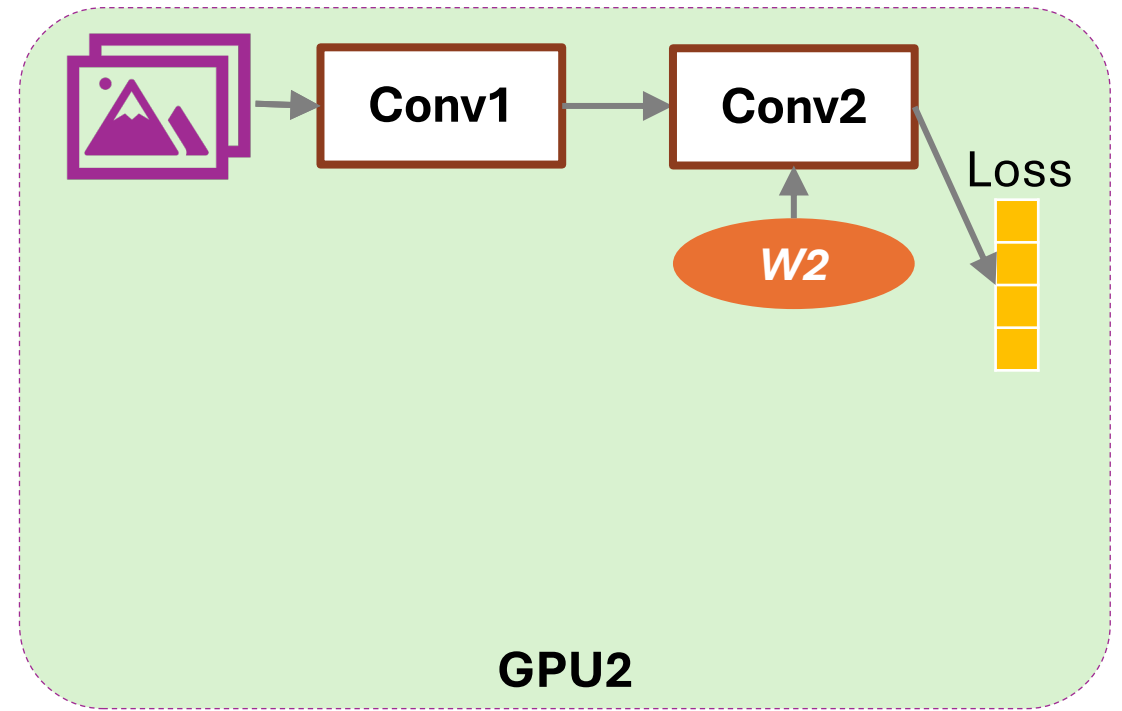
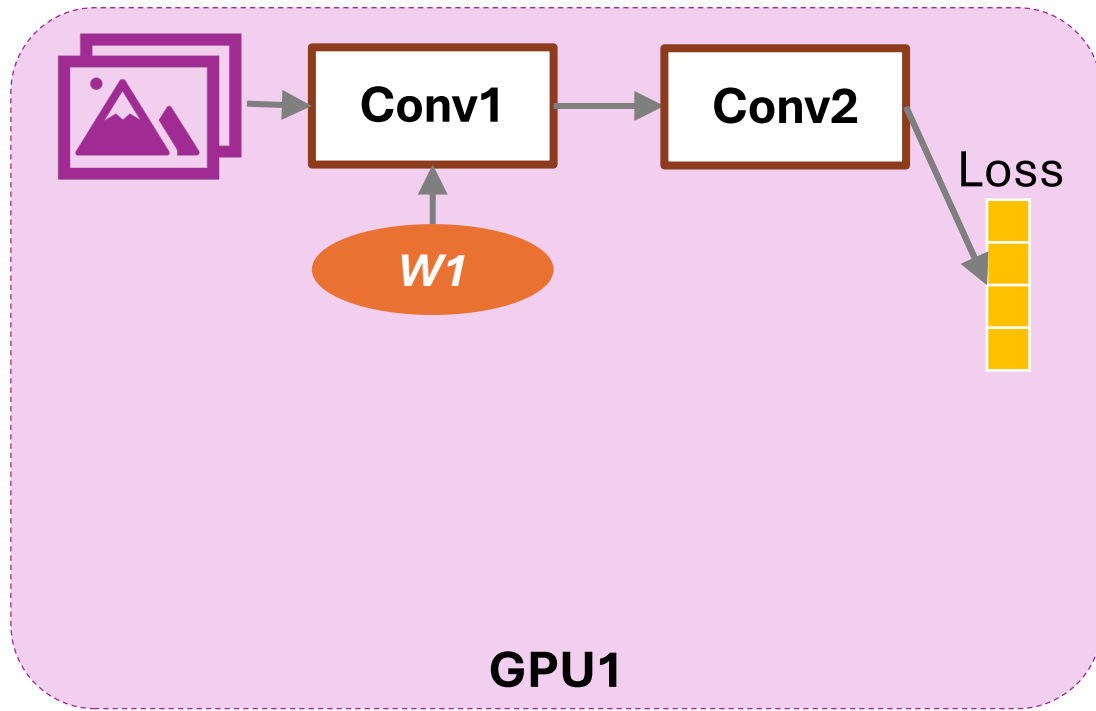
ZeRO Stage 3: Partitioning Parameters

- In ZeRO, model parameters are partitioned across GPUs
- GPUs broadcast their parameters during forward



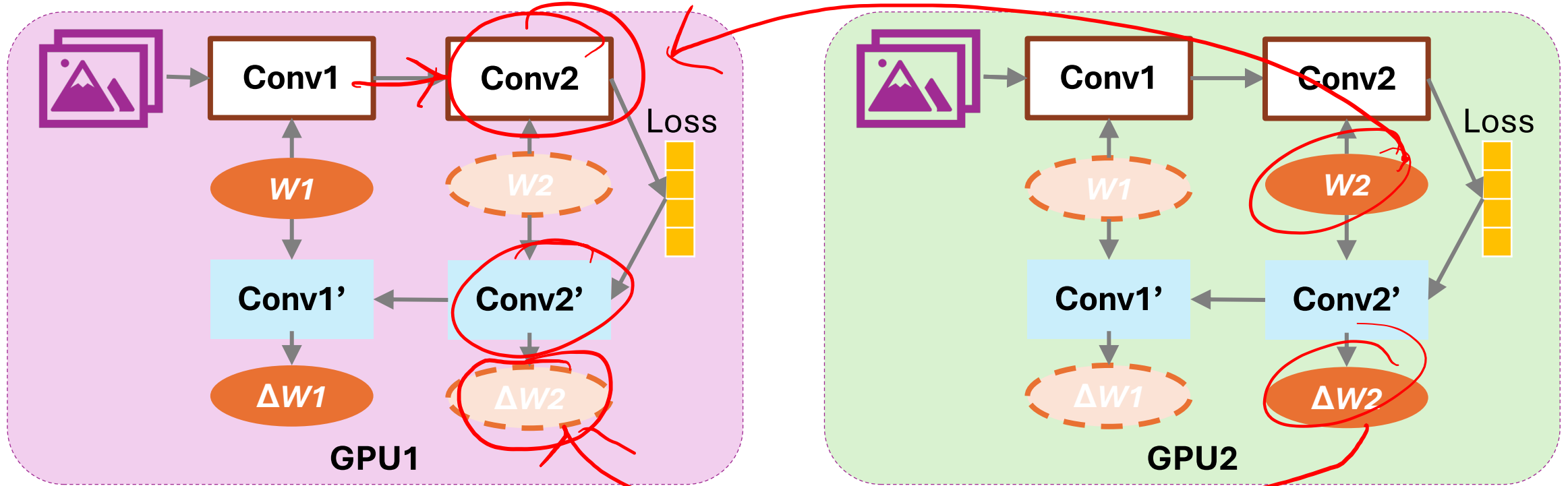
ZeRO Stage 3: Partitioning Parameters

- In ZeRO, model parameters are partitioned across GPUs
- Parameters are discarded right after use

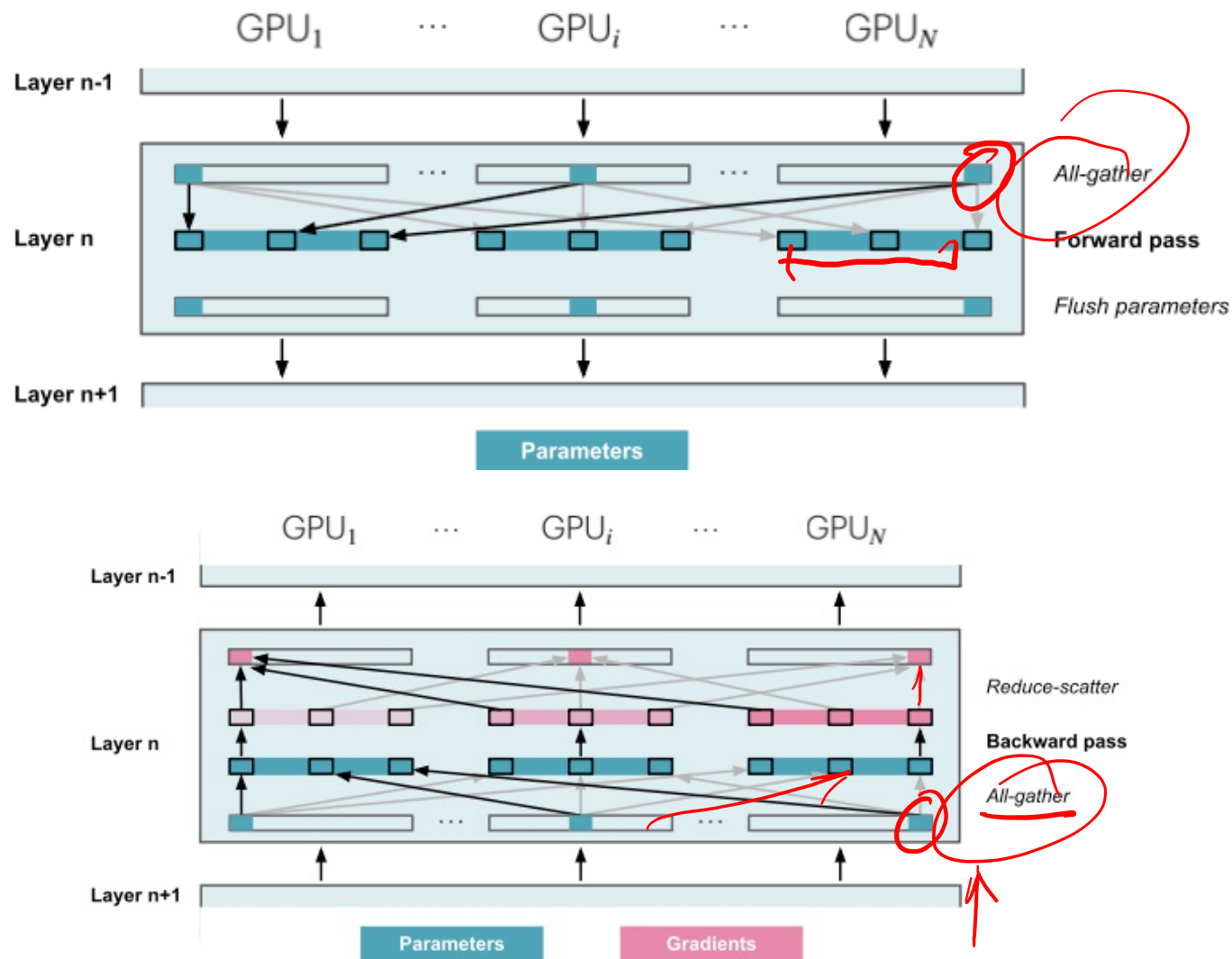


ZeRO Stage 3: Partitioning Parameters

- In ZeRO, model parameters are partitioned across GPUs
- GPUs broadcast their parameters again during backward



ZeRO Stage 3: Partitioning Parameters



ZeRO: Zero Redundancy Optimizer

- ZeRO has three different stages
- Progressive memory savings and communication volume

